Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Chat-First Data Analytics

First Author
University of Somewhere

Abstract

This paper introduces *chat-first data analytics*: an approach in which natural language conversation serves as the primary interface to statistical methods, while rigorous, pre-implemented algorithms perform the actual computation. Unlike approaches where large language models generate statistical code—raising concerns about correctness, data privacy, and reproducibility—chat-first analytics separates the interpretive layer (understanding user intent) from the computational layer (executing validated methods). The language model selects appropriate tools; verified implementations produce the results. We present **prompt2analytics**, a software implementation comprising a foundational econometrics library in pure Rust, exposed through the Model Context Protocol (MCP) for integration with language models. The library covers regression analysis with robust and clustered standard errors, panel data models, instrumental variables, difference-in-differences, discrete choice, and time series methods. All implementations are validated against established R packages. Local deployment via Ollama enables complete data isolation, addressing privacy requirements for sensitive data analysis.

Keywords: chat-first analytics, natural language interface, econometrics, Rust, MCP, local LLM, data privacy.

1. Introduction

The release of ChatGPT in late 2022 changed how many researchers and analysts approach data analysis. Rather than consulting documentation, writing code from scratch, or learning domain-specific syntax, users began asking questions in plain English: "run a regression of wages on education and experience," "test whether these two groups have different means," or "fit an ARIMA model to this time series." While natural language interfaces to statistical software have been explored for decades—from early database query systems (Androutsopoulos, Ritchie, and Thanisch 1995) to commercial tools like SPSS and SAS—LLMs represent a qualitative advance in flexibility and accessibility. This shift promises to lower barriers to

data analysis but also raises significant concerns about accuracy, reproducibility, and data privacy.

This paper introduces the concept of *chat-first data analytics*: an approach that applies established tool-augmented language model techniques (Schick, Dwivedi-Yu, Dessì, Raileanu, Lomeli, Zettlemoyer, Cancedda, and Scialom 2023; Yao, Zhao, Yu, Du, Shafran, Narasimhan, and Cao 2023) to statistical and econometric analysis. Natural language conversation serves as the primary interface to statistical methods, while rigorous, pre-implemented algorithms perform the actual computation. The central insight is that language models excel at understanding intent and selecting appropriate methods, but should not be trusted to generate correct statistical code. By separating the interpretive layer (what does the user want?) from the computational layer (how do we compute it correctly?), chat-first analytics can capture the accessibility benefits of conversational interfaces while maintaining the reliability that scientific work demands.

1.1. The current landscape

The use of large language models (LLMs) for data analysis has grown rapidly (Nejjar, Zacharias, Stiehle, and Weber 2024; Sun, Han, Jiang, Qi, Sun, Yuan, and Huang 2025b; Korinek 2025). The appeal is clear: natural language eliminates the need to learn programming syntax, allowing domain experts to describe analytical goals directly.

The pitfalls, however, are substantial. LLM-generated code may run without error but compute the wrong quantity—requesting "clustered standard errors" might yield robust errors instead, or a panel model might omit fixed effects. Ludwig, Mullainathan, and Rambachan (2024) emphasize the risks of "training leakage" and the difficulty of validating generated code. Users who lack the expertise to write code often lack the expertise to audit it.

Two additional concerns dominate: *data privacy*—sensitive information flows to third-party servers with unclear policies—and *reproducibility*— conversational sessions are ephemeral, with the same prompt yielding different code as models update.

1.2. The opportunity

Despite these concerns, the underlying demand is legitimate and likely to persist. Researchers want to focus on their substantive questions—does education affect wages? did the policy reduce crime? is this treatment effective?—rather than on implementation details. The cognitive burden of translating analytical intent into software-specific syntax is real, and anything that reduces this burden frees mental resources for the questions that actually matter.

Moreover, natural language interfaces enable a style of analysis that code-based workflows discourage: exploratory dialogue. A researcher might begin with "show me the relationship between X and Y," examine the scatterplot, then ask "control for Z," inspect the regression output, and continue "are there outliers driving this?" Each step builds on the previous, with the language model maintaining context across the conversation. This iterative refinement mirrors how experienced analysts actually think about data, but code-based workflows require explicit state management that interrupts the analytical flow.

A further opportunity emerges from reconsidering the computational architecture. When ChatGPT or Claude perform data analysis, they generate Python code that executes in ephemeral virtual machines with limited computational resources. This architecture reflects

the code-generation paradigm: the LLM writes a program, and a sandboxed environment runs it. But if we instead route requests through a structured protocol like MCP, the computational backend need not be constrained by what an LLM can generate or what a sandboxed Python environment can efficiently execute. We can implement statistical methods in compiled languages optimized for numerical performance. Rust, in particular, offers memory safety guarantees enforced at compile time alongside performance characteristics comparable to C and Fortran—the traditional languages of scientific computing. The MCP architecture thus enables high-performance analytics that are difficult to achieve under the code-generation paradigm.

The opportunity, then, is to design systems that preserve the accessibility and interactivity of conversational interfaces while addressing the concerns of correctness, privacy, reproducibility, and performance. Recent work has begun exploring this direction: Chen, Shen, and Ma (2025) develop an "Econometrics AI Agent" that augments LLMs with domain knowledge to improve completion rates on econometric tasks, while Hong, Lin, Liu, Liu, Wu, Li, Chen, Zhang, Wang, Zhang, Zhang, Yang, Zhuge, Guo, Zhou, Tao, Wang, Tang, Lu, Zheng, Liang, Fei, Cheng, Xu, and Wu (2025) present a general-purpose "Data Interpreter" agent. Our approach differs by emphasizing pre-validated implementations and local execution rather than enhanced code generation. This is the goal of chat-first data analytics.

1.3. Chat-first data analytics

We define *chat-first data analytics* as an approach that extends the natural language interface paradigm—developed for database queries, data visualization, and general-purpose AI assistants—to statistical and econometric analysis. This approach has two defining properties and one important design choice:

1. **Natural language as the primary interface.** Users describe their analytical goals in plain language rather than programming syntax. The system interprets requests, selects appropriate methods, and explains results conversationally.

2. **Pre-implemented, verified statistical methods.** The language model does not generate statistical code. Instead, it invokes pre-built, tested implementations of standard methods. Correctness is established once, through validation against reference implementations, rather than verified anew for each user request.

3. **Local data processing** (design choice). In our implementation, bulk data never leaves the user's machine; the language model receives only metadata and aggregated results. This is not definitionally required for chat-first analytics—one could build such a system with cloud-based computation—but we consider it essential for addressing the privacy concerns that motivate much of this work.

This architecture addresses each concern identified above. Correctness is ensured by using validated implementations rather than generated code. Privacy is protected by keeping data local. Reproducibility is achieved because the same method invocation produces identical results—the stochastic element of the language model affects only interpretation, not computation.

The chat-first paradigm differs from both traditional statistical software and from naive LLM-assisted analysis. Unlike R, Stata, or Python, chat-first systems do not require users to learn

domain-specific syntax. Unlike asking ChatGPT to write code, chat-first systems do not trust the language model with numerical computation. The language model serves as a translator—converting natural language intent into structured method calls—while verified software performs the actual analysis. This approach builds on advances in tool-augmented language models (Schick *et al.* 2023; Qin, Liang, Ye, Zhu, Yan, Lu, Lin, Cong, Tang, Qian, Zhao, Hong, Tian, Xie, Zhou, Gerstein, Li, Liu, and Sun 2024; Liu, Huang, Zeng, Hao, Yu, Li, Wang, Gan, Liu, Yu, Wang, Wang, Ning, Hou, Wang, Wu, Wang, Liu, Wang, Tang, Tu, Shang, Jiang, Tang, Lian, Liu, and Chen 2025), which have demonstrated that LLMs can reliably select and invoke external tools when provided with appropriate schemas (Qu, Dai, Wei, Xu, Cai, Wang, Yin, and Wen 2025).

1.4. This paper

This paper presents **prompt2analytics**, a concrete implementation of chat-first data analytics. The software comprises a foundational econometrics library implemented in pure Rust, exposed through the Model Context Protocol (MCP) for integration with language models. Users interact through natural language; the language model interprets requests and generates structured tool calls; the Rust engine executes validated statistical methods; and results flow back through the language model for explanation.

The implementation provides methods across the core areas of applied econometrics: regression analysis with heteroskedasticity-consistent and clustered standard errors, panel data models including high-dimensional fixed effects, instrumental variables estimation, difference-in-differences, discrete choice models, time series analysis, and standard hypothesis tests. All implementations are validated against established R packages to ensure numerical accuracy.

The remainder of the paper is organized as follows. Section 2 elaborates the design philosophy underlying chat-first analytics and relates it to prior work on natural language interfaces, LLM-based agents, and AutoML. Section 3 describes the software architecture of **prompt2analytics**, including the core library organization, user interfaces, and communication flow. Section 4 demonstrates the chat-first workflow through worked examples. Section 5 presents validation results comparing **prompt2analytics** to established R packages, and Section 6 provides detailed performance benchmarks. Section 7 discusses local LLM deployment for complete data isolation. Section 8 concludes with a discussion of limitations and future directions.

1.5. The statistical software landscape

The landscape for econometrics has long been dominated by R (R Core Team 2024), Stata, and increasingly Python. R has developed a rich ecosystem with over 200 econometrics-related packages (Michelaki and Tsagris 2025). Numerical accuracy remains a concern across platforms (McCullough 1997; McCullough and Vinod 1999), motivating careful validation.

Rust has emerged as a compelling platform for scientific computing, combining compiled-language performance with memory safety guarantees enforced at compile time (Rust Foundation 2024). The ecosystem has matured rapidly, with **polars** for DataFrames (Pola-rs Contributors 2024) and **faer** for linear algebra. However, until now, no foundational econometrics library has existed for Rust—a gap that **prompt2analytics** addresses.

1.6. Software availability

prompt2analytics is available as open-source software under the MIT license, which permits commercial use, modification, and redistribution with attribution. The source code, pre-built binaries, installation instructions, and documentation are available at the project repository. Contributions are welcome via pull requests; the repository includes contribution guidelines and a code of conduct.

`https://github.com/prompt2analytics/prompt2analytics`

Supplementary materials accompanying this paper include:

- The 87 test cases used for LLM tool selection evaluation (Section 7), provided as JSON files with prompts, expected tools, and acceptable alternatives
- Benchmark scripts for both R and Rust implementations, enabling reproduction of performance comparisons (Section 6)
- Validation test suites comparing numerical output against R reference implementations (Section 5)

These materials are located in the repository's `paper/` directory and can be executed using the provided Makefile. Appendix F provides detailed reproduction instructions, version information, and containerized build procedures.

2. Design philosophy

Traditional statistical software requires users to learn domain-specific syntax: R formulas, Stata commands, or Python APIs. This creates a barrier where the user must translate their analytical intent into software-specific instructions. **prompt2analytics** inverts this relationship: users describe their goals in natural language, and the system determines the appropriate statistical method.

2.1. Chat-first analytics

The central design principle is what we term *chat-first analytics*: the primary interface is conversational rather than programmatic. A user might request "estimate the effect of education on wages, controlling for experience and industry, with robust standard errors" without specifying whether this requires OLS, fixed effects, or instrumental variables. The system interprets the request, selects appropriate methods, executes the analysis, and returns results with natural language explanation.

This approach builds on a long history of natural language interfaces for data. Early systems like LUNAR (Woods 1973) demonstrated that users could query databases in English, though these systems were limited to specific domains and required careful phrasing (Androutsopoulos *et al.* 1995). The text-to-SQL paradigm, exemplified by benchmarks like Spider (Yu, Zhang, Yang, Yasunaga, Wang, Li, Ma, Li, Yao, Roman, Zhang, and Radev 2018), has driven substantial progress in translating natural language to structured queries (Li, Luo, Chai, Li,

and Tang 2024). Similarly, systems like NL4DV (Narechania, Srinivasan, and Stasko 2021) enable natural language specification of data visualizations.

Our approach differs from these predecessors in two key respects. First, we target statistical analysis rather than data retrieval or visualization alone—the system must select among econometric methods, not just translate queries to SQL. Second, we leverage large language models as the interpretive layer, providing flexibility that rule-based systems cannot match.

This conversational paradigm offers several advantages. It lowers the barrier to entry for users unfamiliar with econometric software conventions. It allows domain experts to focus on their research questions rather than implementation details. And it enables exploratory analysis through dialogue—users can refine their analysis iteratively based on results and suggestions from the system.

2.2. Positioning this work

prompt2analytics builds on a long tradition of interface innovation in statistical software, from the Wilkinson-Rogers formula notation (Wilkinson and Rogers 1973) to text-to-SQL systems (Yu *et al.* 2018) and AutoML pipelines (Feurer, Klein, Eggensperger, Springenberg, Blum, and Hutter 2015). Rather than inventing new notation, it accepts natural language descriptions of analytical intent—trading the precision of formal DSLs for accessibility.

The contribution is primarily software engineering rather than methodological innovation: (1) providing a foundational Rust econometrics library, (2) exposing implementations through a standardized LLM integration protocol, and (3) demonstrating that chat-first interfaces to validated backends can address correctness and privacy concerns. We do not claim to advance econometric theory; the implementations follow established algorithms validated against R reference packages. Appendix H provides extended discussion of related work on statistical interfaces.

2.3. LLM limitations in chat-first systems

The chat-first paradigm does not replace statistical expertise. Users must still understand their data, formulate sensible research questions, and interpret results critically. The system assists with method selection and execution, not with research design or causal identification.

A related concern is the handling of causal language. Users may request analyses using causal terminology ("estimate the effect of X on Y") without satisfying identification assumptions. The system does not validate whether causal claims are supported by the research design; it executes the requested statistical procedure and returns results. LLM explanations may use causal language that echoes user prompts even when the underlying analysis provides only associational evidence. Users bear responsibility for ensuring that causal interpretations are warranted by their identification strategy, not merely by the statistical method employed.

2.4. LLM as method selector

Large language models serve as the interpretive layer between user intent and statistical execution. The LLM parses natural language requests, selects appropriate methods from the toolkit, generates structured tool calls, and interprets results. This builds on the tool-augmented LLM paradigm (Yao *et al.* 2023; Schick *et al.* 2023).

A key distinction from code-generation approaches is that our system does not ask the LLM to write statistical code—studies show LLMs generate incorrect code frequently (Nejjar *et al.* 2024). Instead, we provide pre-implemented, verified methods as callable tools. The LLM's constrained role (method selection and parameter specification) enables smaller models to perform adequately (Section 7).

Table 1: Comparison with existing LLM-based data analysis systems

Feature	**prompt2analytics**	Econometrics AI	Data Interpreter	LAMBDA
Approach	Tool selection	Enhanced code gen	Code gen + exec	Agent orchestration
Computation	Pre-validated Rust	LLM-generated	LLM-generated	LLM-generated
Local execution	Yes (Ollama)	No	No	No
Correctness	Validated vs R	Code review	None	None

Table 1 contrasts **prompt2analytics** with related systems (Chen *et al.* 2025; Hong *et al.* 2025; Sun, Han, Jiang, Qi, Sun, Yuan, and Huang 2025a)—all of which generate and execute code, whereas our system routes requests to pre-validated implementations.

2.5. Why Rust

Rust compiles to native code with C-level performance while guaranteeing memory safety at compile time—eliminating buffer overflows, use-after-free, and data races that can cause incorrect results in numerical software. The ecosystem provides **polars** for data manipulation, **ndarray** for arrays, and **faer** for linear algebra. A single codebase compiles to standalone binaries, WebAssembly for browsers, and shared libraries for language integration.

2.6. MCP as the bridge

The Model Context Protocol (MCP) provides the communication layer between LLMs and the analytics engine (Anthropic 2024).[1] MCP defines a standardized way for language models to discover available tools, invoke them with structured parameters, and receive structured results.

Each statistical method is exposed as an MCP tool with a defined schema specifying required and optional parameters. For example, the OLS tool accepts a dataset identifier, dependent variable, independent variables, and options for robust standard errors. The LLM generates tool calls conforming to this schema, and the MCP server routes them to the appropriate Rust implementation.

This architecture decouples the language understanding layer from the computation layer. The LLM can be swapped (Claude, GPT, Ollama) without changing the analytics engine. Conversely, the statistical implementations can be updated or extended without modifying the LLM integration.

[1]MCP focuses on model-to-tool integration. Complementary protocols address agent-to-agent communication: Google's Agent2Agent (A2A) protocol (Google 2025) enables discovery and task delegation between AI agents, while IBM's Agent Communication Protocol (ACP) (IBM Research 2025) provides lightweight HTTP-native messaging. A2A and ACP merged under Linux Foundation governance in 2025. For single-agent tool invocation—the pattern used in **prompt2analytics**—MCP remains the dominant standard, with adoption by OpenAI, Microsoft, and Google alongside Anthropic.

2.7. Data locality and privacy

A key design principle is that bulk data processing occurs locally. When a dataset is loaded, only metadata is transmitted to the LLM: column names, data types, row counts, and null percentages. The LLM uses this schema information to formulate appropriate tool calls. Statistical computations—matrix operations, parameter estimation, inference—execute entirely on the local machine, with only aggregated results (coefficients, standard errors, test statistics) returned through the LLM.

Tools that transmit data

Users should understand which operations may transmit data beyond metadata and aggregated results:

- *Data preview* (`head`, `tail`, `sample`): Returns actual row values for inspection. Requested sample size determines transmission volume.
- *Data profiling*: May include frequent values from categorical columns, min/max values for numeric columns, and example values for pattern detection.
- *Outlier detection*: Reports specific observations identified as outliers, including their variable values.
- *Error messages*: Failed operations may return the problematic values in error context (e.g., "column 'salary' contains non-numeric value 'N/A' at row 47").

Statistical estimation tools—regression, panel methods, hypothesis tests—transmit only summary statistics (coefficients, standard errors, test statistics, p-values), not individual observations. Residual diagnostics report aggregate properties (normality tests, autocorrelation) rather than observation-level residuals unless specifically requested.

Information leakage through aggregates

Even aggregated statistics can leak sensitive information in certain contexts. Coefficient estimates in small-sample regressions may be influenced by individual observations. Group means with small group sizes (particularly singletons) directly reveal individual values. Summary statistics for highly skewed distributions may identify outlying individuals.

Users working with sensitive data should consider whether aggregated output could enable re-identification or inference about protected individuals. Local LLM deployment via Ollama eliminates these concerns entirely by keeping all data and results on the local machine.

Security considerations

MCP implementations face security challenges common to tool-augmented LLM systems. *Prompt injection* attacks embed malicious instructions in data that the LLM then processes, potentially causing unintended tool invocations. *Tool poisoning* attacks modify tool descriptions to influence LLM behavior (OWASP Foundation 2025).

prompt2analytics mitigates these risks through several design choices:

- Tools have fixed, auditable descriptions embedded in the server binary—external modification is not possible without rebuilding.
- The server executes only pre-defined statistical methods. No arbitrary code execution capability exists; the LLM cannot instruct the server to run shell commands or access files outside loaded datasets.
- Tool calls are logged and can be inspected before execution (in interactive mode) or reviewed in audit logs.
- Dataset access requires explicit loading; the server does not have ambient filesystem access.

Nevertheless, users deploying MCP servers should follow security best practices: avoid loading untrusted data files that could contain injection payloads, review tool calls when processing sensitive data, and prefer local LLM deployment for high-security environments.

The same interface and analytics engine function identically with local or cloud-hosted models, allowing users to select the deployment appropriate to their security requirements.

2.8. Implemented methods

The analytics engine implements standard econometric methods across several categories: regression analysis (OLS with HC0–HC3 robust and clustered standard errors), panel data models (fixed effects, random effects, Hausman test, high-dimensional fixed effects), instrumental variables (2SLS with first-stage diagnostics), causal inference (difference-in-differences, treatment effects), discrete choice models (logit, probit), and time series analysis (ARIMA, VAR, VECM, seasonal decomposition). Full documentation of all methods, including mathematical specifications and implementation details, is available in the package documentation.

2.9. Implementation caveats

Several implemented methods have limitations relative to state-of-the-art practice. Key caveats include: 2SLS reports first-stage F-statistics but lacks Stock-Yogo critical values for weak instruments; DiD assumes the canonical two-group, two-period design and does not implement staggered treatment estimators (Goodman-Bacon 2021; Sun and Abraham 2021); and ARIMA supports non-seasonal models with partial seasonal support.

The implementation focuses on foundational methods; unimplemented categories include Heckman selection, quantile regression, spatial econometrics, GMM, dynamic panel models, matching estimators, and Bayesian methods. Users requiring these should use R or Stata for those analyses. Appendix I provides detailed documentation of method-specific limitations and recommended alternatives.

3. Software architecture

prompt2analytics is organized as a Rust workspace comprising four crates that separate concerns between core computation, user interfaces, and LLM integration. Figure 1 illustrates the overall architecture, showing two distinct pathways: direct access via the CLI, and LLM-mediated access through chat interfaces.

p2a-core Core analytics library implementing all statistical methods in pure Rust. This crate has no external econometrics dependencies; all estimators are implemented from first principles using **ndarray** for matrix operations and **faer** for numerically stable linear algebra. Implementations follow standard textbook algorithms: OLS and robust standard errors follow Wooldridge (2010) and Cameron and Trivedi (2005); panel data methods follow Baltagi (2013); time series methods follow Hamilton (1994); maximum likelihood estimation for discrete choice models uses Newton-Raphson as described in Greene (2018).

p2a-cli Command-line interface exposing the full analytics functionality through hierarchical subcommands. Produces a standalone `p2a` binary.

p2a-mcp Model Context Protocol server exposing 101 tools for LLM integration. Supports multiple transport mechanisms (stdio, HTTP, WebSocket) and maintains session state for dataset persistence.

p2a-dioxus Web application written entirely in Rust using the Dioxus framework, compiling to WebAssembly for browser deployment. Communicates with the MCP server via HTTP.

All user-facing crates depend on **p2a-core**, ensuring that statistical computations are identical regardless of the interface used.

3.1. Core library organization

The **p2a-core** library organizes modules by analytical workflow: linear algebra primitives (`linalg`), regression methods with robust standard errors (`regression`), panel/IV/DiD/discrete choice methods (`econometrics`), 40+ statistical tests (`stats`), time series and forecasting (`forecasting`), clustering and dimensionality reduction (`ml`), and visualization (`visualization`).

A `LinearEstimator` trait provides a consistent interface across OLS, panel models, and IV estimators—implementors provide core results (coefficients, standard errors, residuals); derived statistics (t-values, p-values, confidence intervals) are computed via default implementations. This ensures comparable output across estimators while enforcing the interface at compile time. Unlike R's formula interface, **p2a-core** uses an explicit column-based API (`run_ols(dataset, "y", &["x1", "x2"], ...)`), since the LLM already translates intent to specification.

The `EconError` type provides specific, actionable error messages (singular matrix, perfect multicollinearity, convergence failure, insufficient clusters) that enable the LLM to provide helpful recovery guidance.

3.2. User interfaces

Three interfaces provide access to the core functionality, as shown in Figure 1.

Command-line interface

The CLI uses hierarchical subcommands organized by analytical domain:

```
p2a data load sales.csv --name sales
```

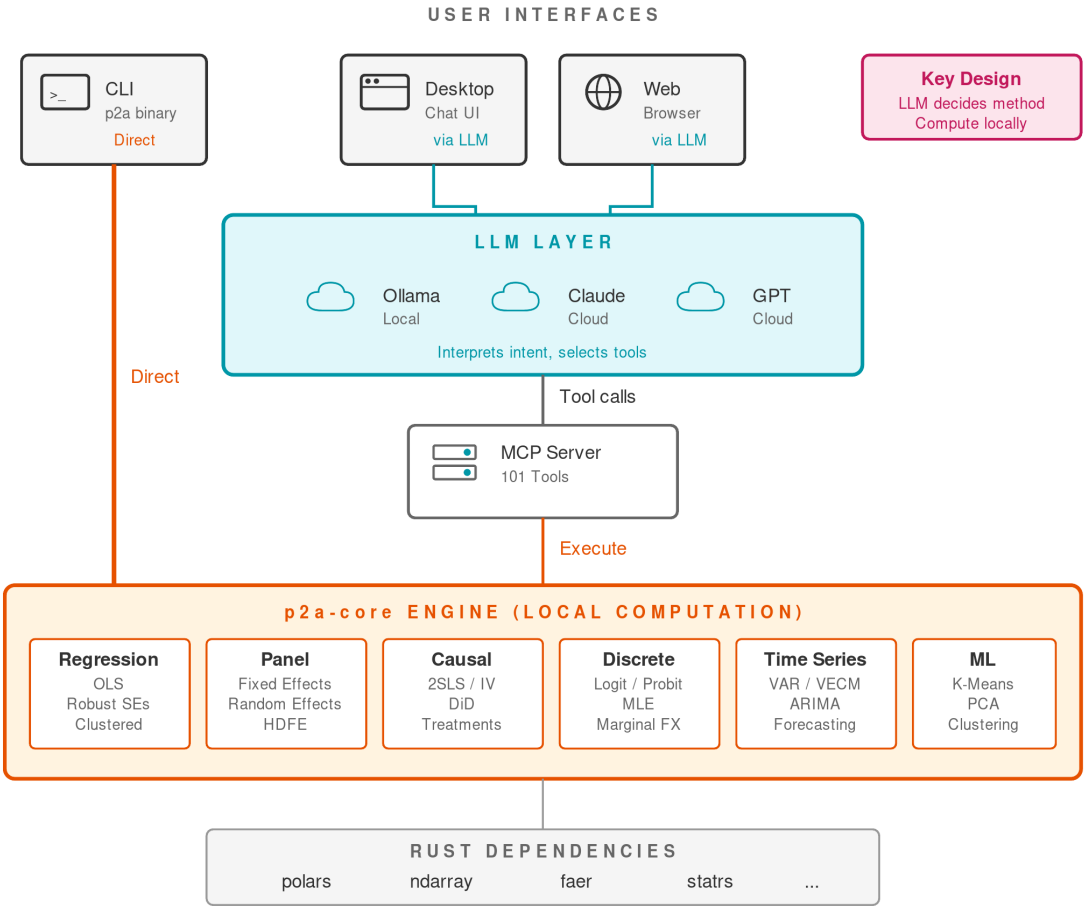

Figure 1: Software architecture of **prompt2analytics**. The CLI provides direct access to the analytics engine (orange path), while chat interfaces route through the LLM layer and MCP server (cyan path). All statistical computation occurs locally in the **p2a-core** engine, which implements methods across six categories using pure Rust dependencies.

```
p2a reg ols sales -y revenue -x advertising budget --robust hc1
p2a panel fe sales -y revenue -x advertising --entity firm --time year
p2a viz scatter sales -x advertising -y revenue -o scatter.png
```

Session management enables dataset persistence across commands and script generation for reproducibility. The `-session` flag records all commands to a JSON file that can be exported as an executable script.

MCP server

The MCP server exposes 101 tools organized into categories that mirror the core library structure:

Category	Examples	Tools
Data management	`load_dataset`, `describe_dataset`	7
Data transformation	`munge_filter`, `munge_join`, `munge_pivot`	19
Regression	`regression_ols`, `regression_diagnostics`	10
Panel data	`panel_fixed_effects`, `hausman_test`	5
Causal inference	`iv_2sls`, `diff_in_diff`	8
Discrete choice	`logit`, `probit`	2
Time series	`ts_arima_fit`, `ts_var`	6
Statistics	various hypothesis tests	30+
Visualization	`viz_scatter`, `viz_histogram_interactive`	7

Each tool has a JSON Schema defining required and optional parameters, enabling LLMs to generate valid tool calls. The schema design follows several principles: required parameters are minimized to reduce LLM errors, sensible defaults are provided (e.g., HC1 robust standard errors), and parameter names use consistent conventions (dataset identifier is always `dataset`, dependent variable is `y_var`). Tool descriptions are written to be interpretable by LLMs, specifying when each tool is appropriate and what alternatives exist. The complete tool schema is available in the supplementary materials (`paper/code/mcp_tool_schema.json`). The server supports multiple transport mechanisms: stdio for direct CLI integration, HTTP for the web application, and WebSocket for streaming results.

Web application

The web interface is implemented entirely in Rust using the Dioxus framework, which compiles to WebAssembly for browser deployment. This pure-Rust approach ensures type safety across the full stack and eliminates the JavaScript build toolchain. The application communicates with the MCP server's HTTP endpoint and supports three LLM providers: Ollama (local), Claude (Anthropic), and GPT (OpenAI). Conversations are persisted via an embedded SurrealDB database on the server side.

3.3. Communication flow

Figure 2 illustrates the interaction pattern for LLM-mediated requests. When a user submits a natural language query:

1. The query is sent to the LLM provider (cloud or local Ollama)

2. The LLM interprets the request and generates a structured tool call with appropriate parameters
3. The MCP server routes the call to the corresponding Rust method in **p2a-core**
4. The core library loads data and performs computation locally
5. Results (coefficients, test statistics, diagnostics) are serialized to JSON and returned through the MCP server
6. The LLM receives the results and formulates a natural language response
7. The response is displayed to the user

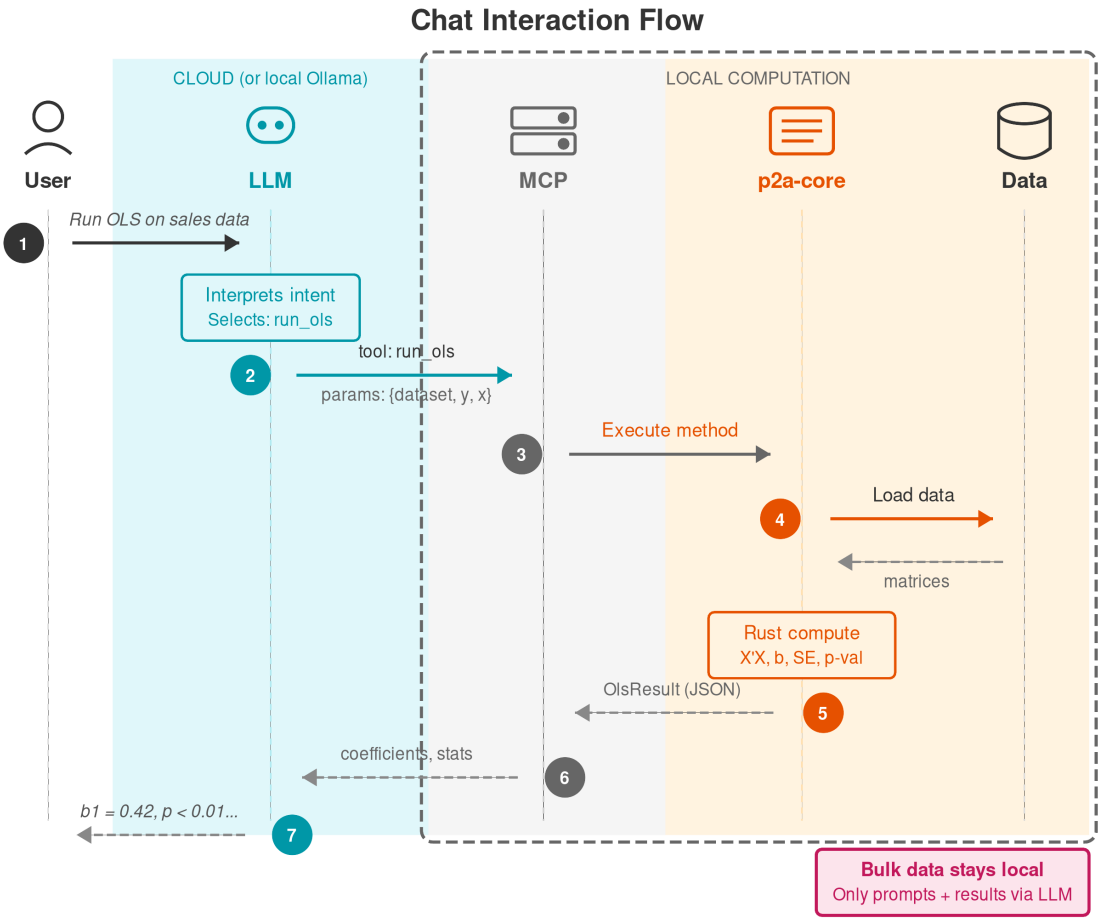

Figure 2: Sequence diagram showing the chat interaction flow. The user's natural language request is interpreted by the LLM, which generates a structured tool call. The MCP server executes the corresponding Rust method, and results flow back through the LLM for natural language explanation. Bulk data processing (shaded region) occurs entirely on the local machine.

This architecture maintains data locality: the full dataset never leaves the user's machine. Only metadata (column names, types), tool parameters, and aggregated results pass through

the LLM. Users requesting data previews should be aware that sample rows will be transmitted; for complete isolation, local LLM deployment via Ollama eliminates all external data transmission.

3.4. Technical design decisions

Several technical choices merit brief explanation.

Numerical precision. All computations use 64-bit floating point (`f64`) throughout, providing approximately 15–17 significant decimal digits. This matches the precision used by R and most established statistical software. Extended precision (128-bit) is not supported; users requiring higher precision for specialized applications (certain financial calculations, cryptographic contexts, or severely ill-conditioned matrices) should use dedicated software.

Conversation persistence. The web application uses SurrealDB, an embedded document database, for storing conversation history and settings. This choice reflects a preference for zero-configuration deployment: the database runs as part of the application binary with no external service required. Analytical datasets are held in memory during sessions and not persisted to the database, limiting storage to lightweight conversation metadata.

Handling unimplemented methods. When users request capabilities outside the implemented set, the LLM's behavior depends on its training. In some cases, it maps the request to the closest available alternative and explains the substitution. In others, it reports that the capability is not available. The system does not explicitly enumerate unimplemented methods; rather, the LLM infers scope from the tool schema. Users seeking methods not in the tool list should verify availability before assuming support.

4. Examples

This section demonstrates practical usage of **prompt2analytics** through two interfaces: the natural language chat interface powered by large language models, and the command-line interface for scripted workflows. Both interfaces access the same underlying Rust analytics engine, offering users flexibility in how they interact with the software.

We use two datasets throughout this section:

- The Grunfeld investment data (Baltagi 2013), a classic panel dataset with 200 observations (10 firms over 20 years) containing firm investment, market value, and capital stock. This dataset is included in the **plm** package and can be loaded via `data("Grunfeld", package = "plm")` in R; we provide it as `grunfeld.csv` in the supplementary materials.
- A simulated cross-sectional dataset demonstrating basic regression analysis with heteroskedasticity.

4.1. Chat interface

The chat interface enables natural language interaction with the analytics engine. Users describe their analysis goals in plain language, and the LLM translates these requests into structured tool calls executed by the MCP server. This interface is particularly valuable for exploratory data analysis and for users less familiar with programming syntax. The examples

in this section were generated using Claude 3.5 Sonnet via the MCP integration; other models (GPT-4o, Llama 3.1 8B, etc.) produce similar results with minor variations in explanation style.

Loading and exploring data

A typical session begins by loading data and examining its structure:

```
User: Load the Grunfeld investment dataset from grunfeld.csv and show me
      what's in it.

I'll load the dataset and describe its contents.

**Dataset loaded: grunfeld**
- Rows: 200
- Columns: 5

| Column  | Type    | Description                        |
|---------|---------|------------------------------------|
| firm    | Integer | Firm identifier (1-10)             |
| year    | Integer | Year (1935-1954)                   |
| inv     | Float64 | Gross investment                   |
| value   | Float64 | Market value of firm               |
| capital | Float64 | Stock of plant and equipment       |

This is a balanced panel dataset with 10 firms observed over 20 years,
commonly used to demonstrate panel data methods.
```

OLS regression with robust standard errors

Cross-sectional regression with heteroskedasticity-consistent standard errors.[2]

```
User: Run a regression of investment on market value and capital stock.
      Use robust standard errors.

Running OLS regression with HC1 robust standard errors...

**OLS Regression Results**
Dependent variable: inv

| Variable    |    Coef |  Std Err |      t | P>|t| |
|-------------|--------:|---------:|-------:|------:|
| (Intercept) | -42.714 |   11.575 | -3.690 | 0.000 |
| value       |   0.116 |    0.007 | 16.967 | 0.000 |
```

[2]HC1 is the default robust standard error estimator when robust SEs are requested. HC1 applies a degrees-of-freedom correction ($n/(n-k)$) to HC0, improving small-sample performance.

```
| capital     |   0.231 |    0.049 |  4.721 | 0.000 |

R-squared: 0.812
Adj. R-squared: 0.810
Observations: 200
Standard errors: HC1 (heteroskedasticity-robust)

Both market value and capital stock are highly significant predictors of
investment. A $1 million increase in market value is associated with
approximately $116,000 more investment, holding capital constant.
```

Panel data: Fixed effects estimation

For panel data, users can request fixed effects estimation in natural language:

```
User: The data has a panel structure. Run a fixed effects regression
      controlling for firm-specific effects.

Running within-transformation fixed effects estimation...

**Fixed Effects (Within) Estimation**
Dependent variable: inv
Entity fixed effects: firm (10 levels)

| Variable |    Coef |  Std Err |      t | P>|t| |
|----------|--------:|---------:|-------:|------:|
| value    |   0.110 |    0.012 |  9.288 | 0.000 |
| capital  |   0.310 |    0.017 | 17.867 | 0.000 |

R-squared (within): 0.767
Observations: 200
Groups: 10
F-statistic: 309.01 (p-value: 0.000)

After controlling for time-invariant firm characteristics, the coefficient
on capital increases from 0.23 to 0.31, suggesting that pooled OLS
underestimated the association between capital stock and investment.
(Note: This change in coefficient reflects removal of omitted variable
bias, not necessarily identification of a causal effect.)
```

Model comparison: Hausman test

Users can request specification tests without knowing the technical details:

```
User: Should I use fixed effects or random effects for this analysis?
```

```
I'll run a Hausman test to help determine the appropriate specification.

**Hausman Test Results**
H0: Random Effects is consistent and efficient
H1: Fixed Effects is consistent; Random Effects is inconsistent

Chi-squared statistic: 0.0003
Degrees of freedom: 2
P-value: 0.9999

**Recommendation**: Random Effects

The test fails to reject the null hypothesis. The near-zero chi-squared
statistic indicates virtually no systematic difference between the FE and
RE coefficient estimates for this dataset.

**Note**: Extreme p-values like 0.9999 warrant interpretation with caution.
Such values can arise from small test statistics (as here), but also from
numerical issues or model misspecification. The Hausman test \citep{Hausman:1978}
assumes both estimators are consistent under the null, which requires correct
functional form, homoskedastic errors, and no omitted variables correlated
with the regressors. The test statistic may be unreliable with small samples,
heteroskedasticity, or serial correlation. Users should treat very high
p-values as suggestive rather than definitive.
```

High-dimensional fixed effects

For datasets with multiple categorical dimensions, users can request multi-way fixed effects:

```
User: Control for both firm and year fixed effects in the regression.


Running two-way fixed effects using alternating projections...

**High-Dimensional Fixed Effects Estimation**
Dependent variable: inv
Fixed effects: firm (10 levels), year (20 levels)

| Variable |    Coef |  Std Err |      t | P>|t| |
|----------|--------:|---------:|-------:|------:|
| value    |   0.118 |    0.019 |  6.137 | 0.000 |
| capital  |   0.358 |    0.054 |  6.579 | 0.000 |

R-squared (within): 0.720
Observations: 200
F-statistic: 217.44 (p-value: 0.000)
```

Regression diagnostics

Users can request diagnostic tests using natural language:

```
User: Check the regression assumptions for the OLS model.

Running regression diagnostics...

**Jarque-Bera Test (Normality of residuals)**
Statistic: 89.57
P-value: 0.000
Conclusion: Residuals deviate significantly from normality.

**Breusch-Pagan Test (Heteroskedasticity)**
Statistic: 55.25
P-value: 0.000
Conclusion: Evidence of heteroskedasticity; robust SEs recommended.

**Durbin-Watson Statistic (Autocorrelation)**
Statistic: 0.358
Interpretation: Positive autocorrelation detected (expected in panel data).

**Variance Inflation Factors**
value: 1.31
capital: 1.31
No multicollinearity detected (VIF < 5).
```

4.2. Command-line interface

The command-line interface provides programmatic access for scripted workflows. Commands follow a hierarchical structure with sessions maintaining state:

```
# Load data and run OLS with robust standard errors
p2a --session analysis.json data load grunfeld.csv --name grunfeld
p2a --session analysis.json reg ols grunfeld \
    -y inv -x value capital --robust hc1

# Panel fixed effects
p2a --session analysis.json panel fe grunfeld \
    -y inv -x value capital --entity firm

# Export session to reproducible script
p2a script export analysis.json -o reproduce.sh
```

All commands support JSON output (`-F json`) for programmatic integration. Appendix J provides additional CLI examples including panel estimation, hypothesis tests, and two-way fixed effects.

4.3. Comparison with R

For reference, the equivalent R code for the fixed effects estimation is:

```
library(plm)
data("Grunfeld", package = "plm")

# Fixed effects model
fe_model <- plm(inv ~ value + capital,
                data = Grunfeld,
                index = c("firm", "year"),
                model = "within")
summary(fe_model)

# Two-way fixed effects
fe_twoway <- plm(inv ~ value + capital,
                 data = Grunfeld,
                 index = c("firm", "year"),
                 model = "within",
                 effect = "twoways")
summary(fe_twoway)
```

The **prompt2analytics** implementation produces numerically equivalent results (see Section 5), while offering the conversational interface that enables users unfamiliar with R syntax to perform the same analyses through natural language.

4.4. Multi-step workflows

Real-world analysis involves multiple steps: data inspection, handling issues, and specification refinement. The chat interface maintains context across turns, enabling progressive model refinement guided by user questions. The LLM provides interpretive commentary while all computations execute through validated Rust implementations.

Appendix J provides a detailed multi-step workflow example demonstrating data loading, missingness analysis, initial specification, and refinement with fixed effects—illustrating context preservation and proactive issue identification.

4.5. Error handling and recovery

The system handles common failure modes with explanatory guidance. Two representative examples illustrate the approach.

Multicollinearity detection

```
User: Add years_since_hire to the model alongside tenure.

**Error: Perfect multicollinearity detected**
```

```
The design matrix is rank-deficient. Column `years_since_hire` is a linear
combination of existing columns (correlation with tenure: 1.000).

**Suggested actions:**
1. Remove one of the collinear variables
2. If both are theoretically important, consider which better captures
   your construct of interest
```

Convergence failure

```
User: Fit a logit model with department fixed effects.

**Warning: Convergence issues detected**

**Issue:** Perfect prediction in some groups
- Department "Legal" has 0% bonus rate (23 observations)

**Options:**
1. Drop singleton departments
2. Use a linear probability model (robust to separation)
3. Use penalized likelihood (Firth correction)

Running linear probability model with department FE instead...
```

The system detects problems before estimation fails, explains statistical issues rather than computational symptoms, and provides actionable suggestions. When requests exceed capabilities, it clearly states limitations and points to external tools. Appendix J provides additional error handling examples including small cluster warnings and out-of-scope request handling.

5. Comparison and validation

A statistical software package must demonstrate both correctness and competitive performance. This section compares **prompt2analytics** against established R packages on both dimensions, showing that the Rust implementation achieves substantial speedups—ranging from 2× to over 100× depending on the method—while matching reference implementations to high numerical precision.

5.1. Methodology

Comparisons use synthetic datasets generated with identical data-generating processes in both languages (seed = 42) and classic datasets with published results (Longley for OLS, Grunfeld for panel data). Reference implementations include `lm()`/**sandwich** for OLS, **plm**/**lfe** for panel data, `glm()` for discrete choice, and **forecast** for time series. Each benchmark executes 100 iterations after warmup; all tests run on identical hardware (Intel i7-1260P, 64GB RAM).

Appendix G provides complete details on data generating processes, execution protocol, and software versions.

5.2. Performance comparison

Table 2 summarizes performance results across representative methods and sample sizes. The speedup factors vary considerably by method category and sample size, reflecting differences in algorithmic complexity and the relative weight of interpreter overhead versus numerical computation.

Table 2: Performance comparison: median execution time (μs) and speedup factors for representative methods

Category	Method	n	R	**p2a**	Speedup
Regression	OLS	100	812	42	19×
	OLS	10,000	2,181	924	2.4×
	OLS + HC1	10,000	25,224	1,571	16×
Panel	Fixed effects	100	4,901	27	184×
	HDFE (2-way)	5,000	26,657	1,159	23×
Discrete	Logit	1,000	2,176	401	5.4×
	Probit	1,000	2,760	2,347	1.2×
Time series	ARIMA(1,1,1)	500	4,783	521	9.2×
	MSTL	500	1,853	219	8.5×
ML	K-means ($k = 3$)	1,000	1,274	587	2.2×
	PCA	1,000	246	67	3.7×

Several patterns emerge from these results. First, speedup factors are largest for methods with significant per-call overhead in R. Panel data methods show the most dramatic improvements: fixed effects estimation at $n = 100$ is 184× faster in **prompt2analytics**, reflecting the substantial overhead of **plm**'s formula parsing, factor handling, and data frame manipulation rather than fundamental computational differences. This extreme speedup should be interpreted with caution: it primarily reflects **plm**'s known inefficiencies for small samples, not a general 184× advantage. As sample size increases and matrix operations begin to dominate, speedup factors decrease to more representative levels (5–6× at $n = 10{,}000$). Users should consider the $n = 10{,}000$ results as more indicative of typical production performance.

Second, methods involving expensive mathematical functions show smaller speedups. Probit estimation requires repeated evaluation of the standard normal CDF and PDF; both R (via compiled C code) and Rust incur similar costs for these operations, yielding a modest 1.2× speedup at $n = 1{,}000$. Logit, which uses the computationally simpler logistic function, achieves 5.4× speedup at the same sample size.

Third, robust standard error computation shows consistent speedups (13–16×) across sample sizes. The HC1 estimator requires computing the "meat" matrix $\sum_i \hat{u}_i^2 x_i x_i'$, which benefits from Rust's efficient memory access patterns and SIMD vectorization in the **faer** linear algebra library.

Figure 3 displays the full timing distributions for selected methods at $n = 10{,}000$. Beyond the differences in central tendency, **prompt2analytics** shows substantially lower variance: interquartile ranges are typically 10–100× smaller than in R, reflecting the absence of garbage collection pauses and interpreter overhead. This predictability is valuable for production deployments where consistent response times matter.

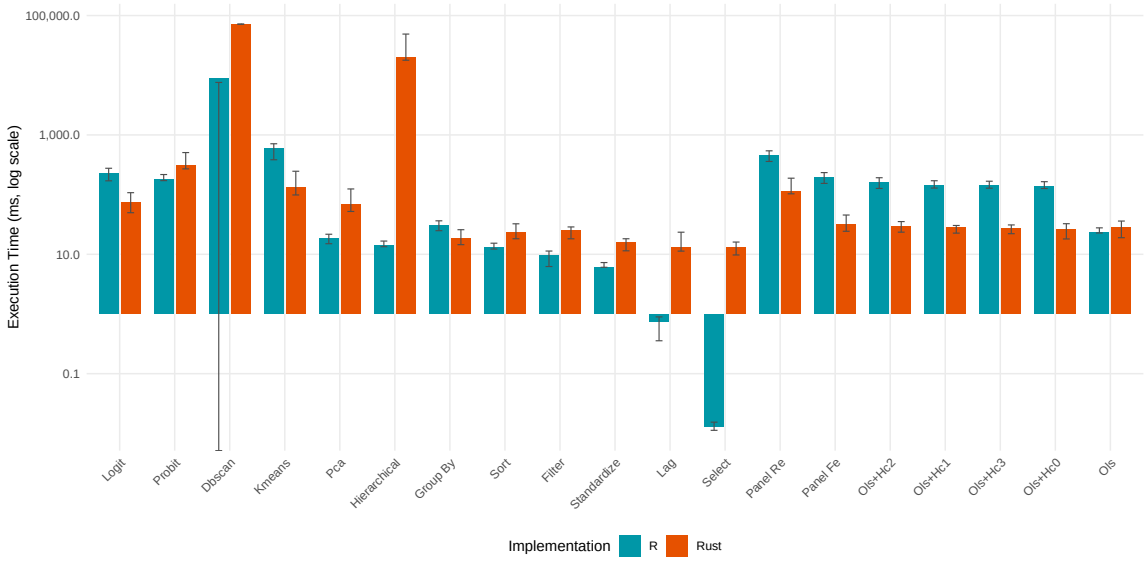

Figure 3: Execution time distributions for R (blue) and **prompt2analytics** (orange) at $n = 10{,}000$. Y-axis uses logarithmic scale. Boxes show interquartile range; whiskers extend to 1.5× IQR.

Memory usage differences are equally striking. For OLS at $n = 10{,}000$, R allocates approximately 3.6 MB (measured via `bench::mark()`) while **prompt2analytics** reports minimal allocation—under 1 KB beyond the pre-allocated data structures—because operations proceed in-place where possible and temporary buffers are reused across iterations. For robust standard errors at the same sample size, R allocates over 10 MB due to intermediate matrix copies; **prompt2analytics** avoids these through careful ownership management. Rust allocations are measured via custom instrumentation of the global allocator during benchmark runs.

5.3. Numerical validation

We validate all methods against reference implementations using sample-size-dependent tolerances (looser for small samples where ill-conditioning amplifies differences, tighter for large samples). The Longley dataset, with its extreme multicollinearity (condition number $> 10^9$), provides a stringent test. Table 3 shows coefficients match R's `lm()` to at least 8 significant digits.

Heteroskedasticity-consistent standard errors (HC0–HC3) are validated against the **sandwich** package. For HC0 and HC1, which differ only by a degrees-of-freedom correction, results match to $< 10^{-8}$. For HC2 and HC3, which involve leverage values $h_{ii} = x_i'(\mathbf{X}'\mathbf{X})^{-1}x_i$, results match to $< 10^{-7}$. The slightly larger tolerance reflects algorithmic differences in leverage

Table 3: OLS validation on the Longley dataset ($n = 16$, $k = 6$)

Coefficient	R `lm()`	**prompt2analytics**	Difference
(Intercept)	−3482.259	−3482.259	$< 10^{-8}$
GNP.deflator	0.01506	0.01506	$< 10^{-8}$
GNP	−0.03582	−0.03582	$< 10^{-8}$
Unemployed	−0.02020	−0.02020	$< 10^{-8}$
Armed.Forces	−0.01033	−0.01033	$< 10^{-8}$
Population	−0.05110	−0.05110	$< 10^{-8}$
Year	1.82915	1.82915	$< 10^{-8}$
R^2	0.9955	0.9955	$< 10^{-4}$

computation: **sandwich** extracts leverage from the QR decomposition used internally by `lm()`, while **prompt2analytics** computes the hat matrix diagonal directly via matrix multiplication. Both approaches are numerically valid; the discrepancy arises from different accumulation of floating-point rounding errors rather than any methodological difference. For applied use, differences at the 10^{-7} level are negligible.

Panel data methods are validated using the Grunfeld dataset (10 firms, 20 years, $n = 200$). Fixed effects coefficients match **plm** to $< 10^{-6}$, and the within R^2 matches to four decimal places. The Hausman test statistic for choosing between fixed and random effects matches to $< 10^{-4}$. Two-way high-dimensional fixed effects, implemented using the MAP algorithm (Gaure 2013), matches **lfe**'s `felm()` to $< 10^{-5}$.

Beyond these flagship methods, the validation suite covers over 50 methods with more than 100 individual test cases. Statistical tests (t-tests, ANOVA, chi-squared, Fisher's exact, Wilcoxon, Shapiro-Wilk, Kolmogorov-Smirnov) are validated against R's **stats** package. Discrete choice models (logit, probit) match `glm()` coefficients and standard errors. Time series methods (VAR, ARIMA) match the **vars** and **forecast** packages. Clustering algorithms match `kmeans()` cluster assignments and within-cluster sum of squares. All validation tests are integrated into continuous integration and can be run via `cargo test -p p2a-core`.

5.4. Sources of performance differences

The performance advantages of **prompt2analytics** stem from several architectural choices inherent to Rust and the library design.

First, Rust's ownership system enables stack allocation and in-place mutation without garbage collection. Matrix operations can reuse memory rather than allocating new buffers, and there are no GC pauses during computation. This is particularly impactful for iterative algorithms (HDFE, MLE) where R would allocate temporary objects on each iteration.

Second, the **faer** linear algebra library provides highly optimized implementations that leverage SIMD instructions on modern processors. While R with OpenBLAS also uses SIMD for matrix multiplication, the overhead of crossing the R-C boundary for each operation accumulates.

Third, the column-based API eliminates formula parsing. R's model formula interface (`y`

`~ x1 + x2`) involves parsing, factor expansion, and model matrix construction—operations that can dominate execution time for small datasets. **prompt2analytics** receives pre-specified column names directly from the MCP tool call.

Fourth, the single-language design avoids the overhead of data serialization between components. In typical R workflows, data moves between R, C, and potentially Fortran; each transition involves memory copies and type conversions. **prompt2analytics** maintains data in native Rust structures throughout.

These advantages diminish as sample size increases and matrix algebra dominates. For very large datasets ($n > 100{,}000$), speedup factors converge toward 2–3× as both implementations spend most time in optimized BLAS routines. The largest advantages appear for small-to-medium datasets ($n = 100$ to 10,000) and for methods with significant per-call overhead.

5.5. Feature comparison

Performance is one dimension of software quality; feature completeness and ecosystem integration are equally important for applied work. Table 4 compares **prompt2analytics** against established R packages across dimensions beyond execution speed.

For core panel methods, **prompt2analytics** provides comparable coverage to specialized packages; gaps include advanced standard errors (Driscoll-Kraay, Conley spatial) and staggered DiD estimators. The distinguishing features are the natural language interface and fully local execution without R dependencies. Performance advantages become practically relevant for simulation studies, bootstrap procedures, and deployment contexts where R installation is impractical; for typical one-off analyses, speed differences are imperceptible.

5.6. Limitations

Several caveats apply to these comparisons. First, we compare against specific R packages chosen for their widespread use in applied econometrics; the comparison necessarily reflects a snapshot of rapidly evolving software. Second, benchmarks use synthetic data with simple structure; real datasets with missing values, factors, or complex hierarchies may show different relative performance. Third, R's flexibility (formula interface, extensive diagnostics, integration with the tidyverse ecosystem) provides value not captured in raw timing comparisons or feature checklists.

6. Performance benchmarks

This section presents detailed performance benchmarks comparing **prompt2analytics** against established R packages across approximately 75 method configurations spanning regression, econometrics, machine learning, and forecasting. Benchmark methodology follows Section 5; see Appendix G for complete details.[3]

6.1. Benchmark results

We present benchmark results across five method categories: regression, panel econometrics,

[3] A notable omission is **fixest** (Bergé 2018), which likely exceeds **prompt2analytics** for HDFE problems. All performance claims compare against **plm** and **lfe**, not **fixest**.

Table 4: Feature comparison with established R packages

Feature	**p2a**	**fixest**	**lfe**	**plm**
Panel data methods				
Within (FE) estimator	✓	✓	✓	✓
Random effects (GLS)	✓	—	—	✓
High-dimensional FE	✓	✓	✓	—
Multi-way clustering	2-way	n-way	2-way	2-way
Hausman test	✓	—	—	✓
Standard errors				
HC0–HC3 robust	✓	✓	✓	✓
Driscoll-Kraay	—	✓	—	✓
Newey-West HAC	—	✓	✓	✓
Conley spatial	—	✓	✓	—
Causal inference				
Difference-in-differences	✓	✓	—	—
Sun-Abraham (staggered)	—	✓	—	—
IV/2SLS	✓	✓	✓	✓
Weak instrument tests	F-stat	✓	✓	—
Diagnostics & output				
Coefficient plots	—	✓	—	—
Regression tables (export)	JSON	LaTeX/HTML	LaTeX	LaTeX
Marginal effects	AME	✓	—	—
Formula interface	—	✓	✓	✓
Integration				
LLM/MCP interface	✓	—	—	—
Local execution (no R)	✓	—	—	—
tidyverse compatible	—	✓	—	✓

discrete choice, machine learning, and data munging. Sample sizes range from $n = 100$ to $n = 100{,}000$ observations.

Table 5 presents benchmark results at $n = 100{,}000$ observations, where computational costs dominate startup overhead. **prompt2analytics** demonstrates substantial speedups for compute-intensive methods: Panel Fixed Effects (6.1×), OLS with robust standard errors (5.1–5.5× for HC0–HC3), K-Means clustering (4.5×), Panel Random Effects (4.0×), and Logit regression (3.1×). Group-by aggregation shows a 1.6× speedup. These speedups at $n = 100{,}000$ represent the most reliable performance comparison, as computational costs dominate language overhead at this scale. Results at smaller sample sizes (not shown) exhibit larger speedup factors that partly reflect **plm**'s known inefficiencies at small n rather than fundamental Rust performance advantages.

Table 5: Performance benchmark results at $n = 100{,}000$ (median execution time). Speedup = R time / Rust time. Methods with speedup $> 1\times$ are faster in Rust; methods $< 1\times$ are faster in R, typically due to CLI startup overhead dominating trivial computations.

Category	Method	n	R (ms)	Rust (ms)	Speedup
Discrete	Logit	100,000	230.1	75.0	3.1×
Discrete	Probit	100,000	180.8	306.4	0.6×
ML	Kmeans	100,000	592.4	131.7	4.5×
ML	Pca	100,000	18.1	68.5	0.3×
ML	Dbscan	100,000	8762.3	71707.0	0.1×
Munging	Group By	100,000	30.3	18.6	1.6×
Munging	Sort	100,000	12.8	23.4	0.6×
Munging	Filter	100,000	9.4	24.8	0.4×
Munging	Standardize	100,000	6.1	15.9	0.4×
Munging	Lag	100,000	0.7	12.9	0.1×
Munging	Select	100,000	0.0	12.8	0.0×
Panel	Panel Fe	100,000	194.8	31.7	6.1×
Panel	Panel Re	100,000	451.1	114.0	4.0×
Regression	Ols+Hc2	100,000	159.8	29.3	5.5×
Regression	Ols+Hc0	100,000	137.2	26.0	5.3×
Regression	Ols+Hc3	100,000	139.6	27.2	5.1×
Regression	Ols+Hc1	100,000	142.3	27.9	5.1×
Regression	Ols	100,000	23.1	28.0	0.8×

Methods showing R outperforming Rust warrant detailed explanation, as these apparent "slowdowns" illuminate the performance characteristics of both approaches.

Data manipulation operations (filter, mutate, group-by aggregation) show modest speedups or near-parity rather than the dramatic gains seen in statistical methods. This reflects R's mature, highly optimized implementations for basic operations: `dplyr()` leverages vectorized C++ code that minimizes overhead for common transformations. The **polars** library used in **prompt2analytics** provides competitive but not superior performance for these operations. Users should not expect data wrangling to be dramatically faster than established R workflows.

PCA showing a 0.3× speedup (i.e., R is faster) reflects R's use of highly optimized LAPACK

routines for eigenvalue decomposition. The Rust implementation via **faer** prioritizes numerical stability and memory safety over raw speed for this operation. For PCA as a dimensionality reduction step in larger workflows, this difference is typically negligible; for workflows dominated by repeated PCA computations, R's `prcomp()` may be preferable.

Probit regression uses Newton-Raphson optimization in our implementation, while R's `glm()` benefits from decades of refinement in iteratively reweighted least squares (IRLS). The difference represents implementation maturity rather than fundamental language performance.

OLS without robust standard errors is sufficiently fast in both languages that CLI startup overhead dominates computation time at moderate sample sizes. The "slowdown" disappears when measured as pure computation time (excluding process initialization).

The general pattern is that **prompt2analytics** excels at computationally intensive iterative methods (panel fixed effects, robust standard errors, HDFE) where Rust's compile-time optimizations and memory efficiency compound across iterations. For single-pass operations dominated by optimized linear algebra, R's BLAS bindings provide comparable or superior performance.

Scaling behavior

Figure 4 shows the distribution of speedup factors across all benchmarked methods at $n = 100{,}000$ observations.

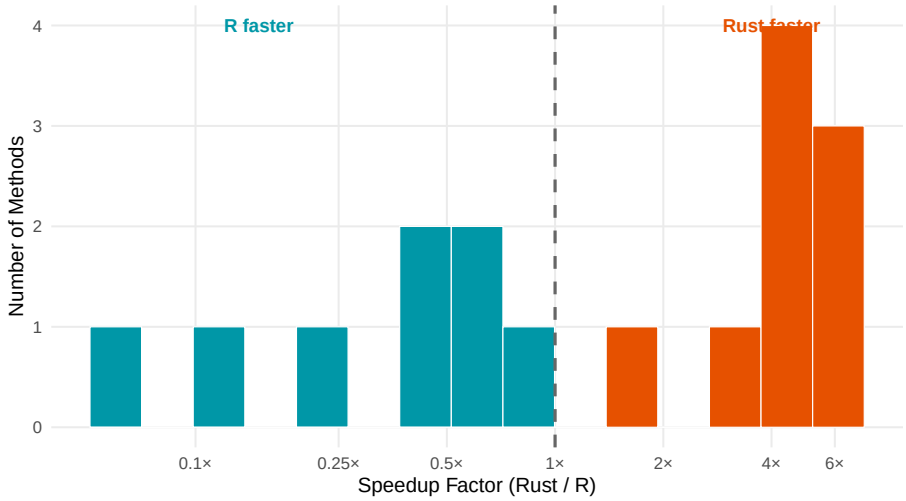

Figure 4: Distribution of speedup factors across all benchmarked methods at $n = 100{,}000$ observations. The dashed line indicates parity ($1\times$). Methods to the right show Rust outperforming R, while methods to the left show R faster—typically due to CLI startup overhead dominating trivial computations.

Figure 5 provides a detailed view of speedup factors by individual method.

Figure 6 shows execution time comparisons between R and Rust implementations.

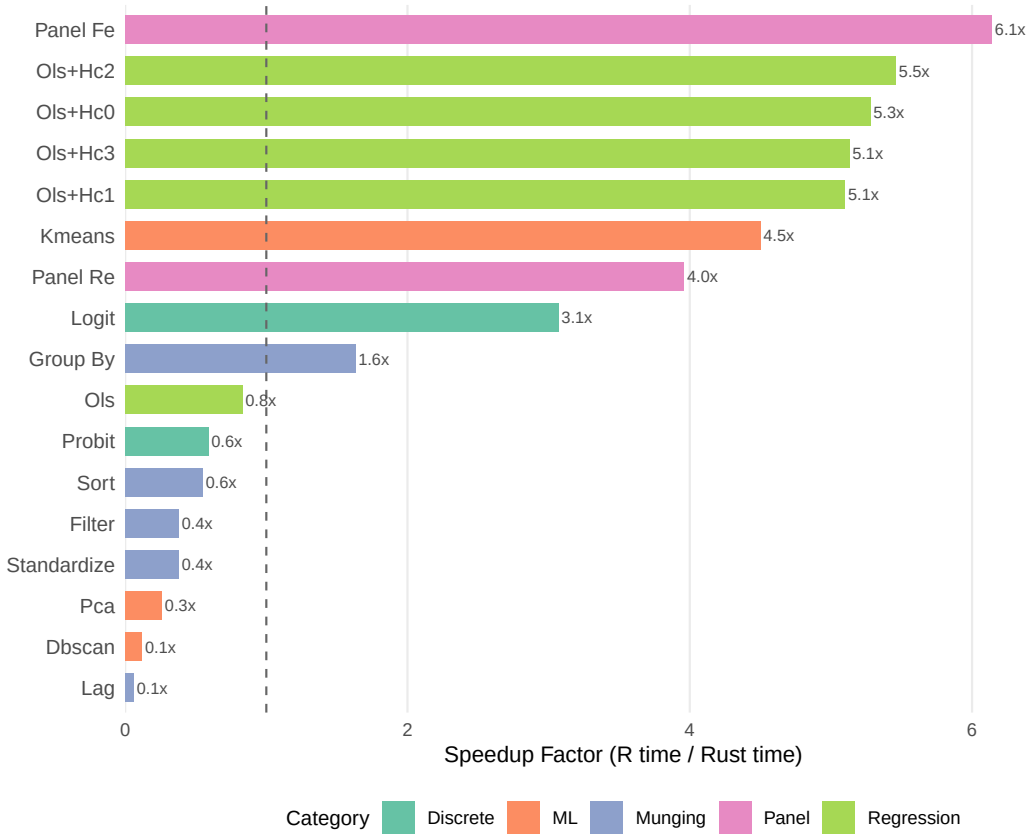

Figure 5: Rust/R speedup factors by method at $n = 100{,}000$ observations. The vertical line at 1× indicates parity. Bars extending right indicate Rust faster; bars extending left indicate R faster. Compute-intensive iterative methods (Panel FE, OLS+robust SEs, K-Means, Panel RE, Logit) show 3–6× speedups. Methods below 1×—including PCA (0.3×), basic data operations, and Probit—reflect either R's highly optimized BLAS/LAPACK libraries or CLI startup overhead dominating trivial computations. Gray scale distinguishes method categories for grayscale reproduction.

Figure 6: Execution time comparison between R and Rust implementations at $n = 100{,}000$ observations (log scale). Error bars show min/max execution times across 100 iterations. For methods where computation time exceeds startup overhead, Rust demonstrates consistent performance advantages.

Discussion

Performance differences reflect Rust's ownership model (reduced heap allocation), SIMD vectorization via **faer**, compiled vs. interpreted overhead, and column-based API (no formula parsing). Speedup factors decrease with sample size as matrix operations dominate.

7. Fully local deployment

While cloud-hosted language models offer convenience and capability, many use cases require complete data isolation: proprietary business data, personally identifiable information under privacy regulations, or air-gapped environments. This section describes the fully local deployment configuration, where all components—including the language model—run on the user's machine.

7.1. Architecture

Figure 7 illustrates the local deployment architecture. The key difference from cloud deployment is that the LLM runs locally via Ollama (Ollama Inc. 2024), eliminating all external network communication. User queries, tool calls, and results never leave the local machine.

The local configuration requires:

- Ollama runtime with a function-calling-capable model
- **prompt2analytics** MCP server

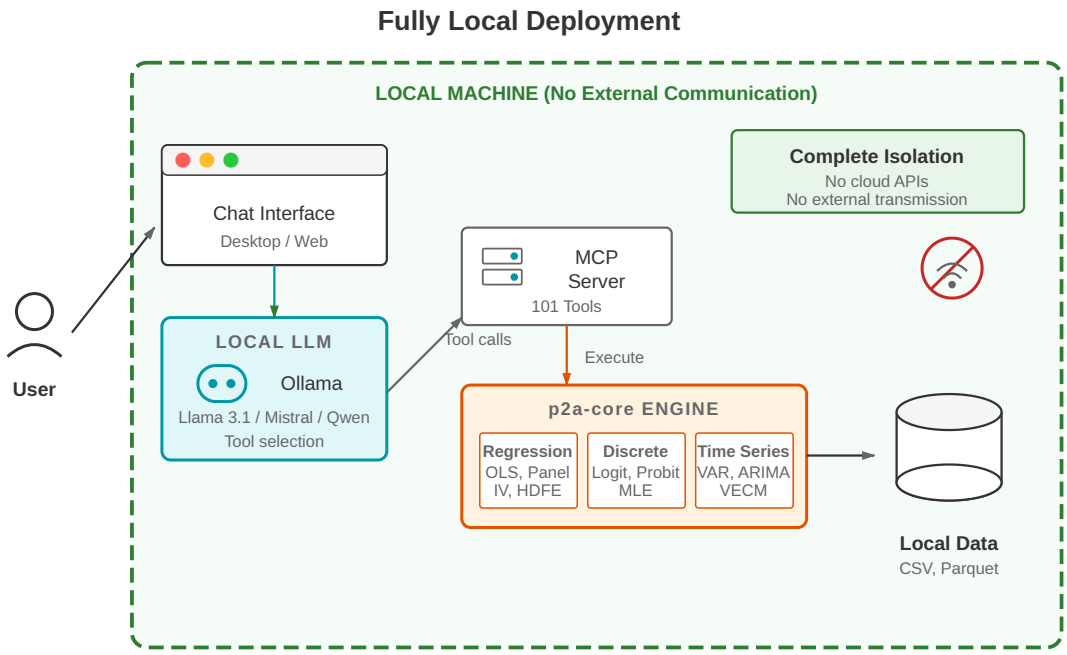

Figure 7: Fully local deployment architecture. All components run on the user's machine: Ollama hosts the language model, which communicates with the MCP server to execute analytics methods in p2a-core. No data or queries are transmitted externally.

- Sufficient RAM for both the model and data processing

7.2. Tool-calling capabilities in local models

The chat-first architecture reduces capability requirements: models need only parse requests, select tools from the MCP schema, generate parameters, and interpret results—not generate statistical code from scratch. Research shows fine-tuned small models can match larger models on targeted tool selection (Schick *et al.* 2023; Patil, Zhang, Wang, and Gonzalez 2023), making this particularly relevant for chat-first analytics with its well-defined schema.

7.3. Tool selection evaluation

We evaluated LLM performance on 87 test cases spanning regression, panel data, causal inference, discrete choice, time series, hypothesis testing, machine learning, and visualization. Each case presents a natural language prompt; the model must select the appropriate tool from the **prompt2analytics** schema. Responses are scored as exact (correct tool), acceptable (valid alternative), category (correct domain, wrong method), or failed. Accuracy is computed as (exact + acceptable) / total.

Figure 8 presents overall accuracy by model. The results demonstrate that tool selection—unlike code generation—is achievable even by smaller models. Claude 3.5 Haiku and Qwen 2.5 72B (an open-source model) achieve 100% accuracy. Notably, Ministral 3B, a model small enough to run on consumer hardware with only 6 GB VRAM, achieves 90.8% accuracy, making it viable for local deployment scenarios where hardware constraints preclude larger

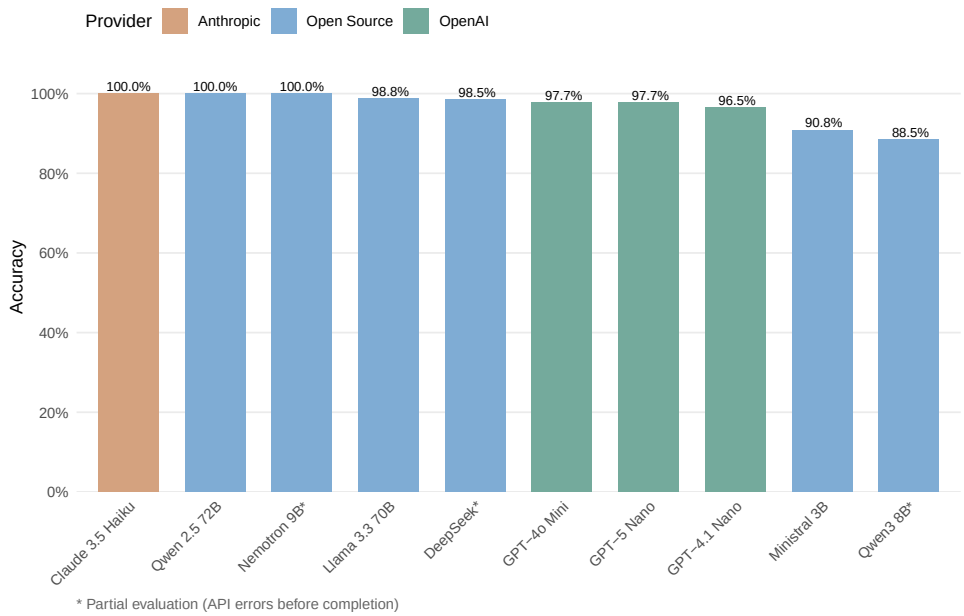

Figure 8: Tool selection accuracy across models. Accuracy is computed as (exact + acceptable matches) / total tests. All models were evaluated on the complete 87-test benchmark. Claude 3.5 Haiku and Qwen 2.5 72B achieve perfect accuracy; even the smallest model (Ministral 3B at 3 billion parameters) exceeds 90%.

models.

Figure 9 shows accuracy by category. Performance is uniformly high across most categories, with errors concentrated in edge cases requiring fine distinctions between related methods. For instance, Ministral 3B shows lower accuracy on time series (66.7%) where distinguishing between VAR, VARMA, and VECM specifications proves challenging for the smaller model. Specific confusion patterns include: selecting VAR when VECM is appropriate (failing to recognize cointegration requirements), choosing ARIMA for multivariate series that require VAR, and defaulting to simpler univariate methods when the prompt describes multiple interrelated series. These errors reflect the subtle distinctions in time series econometrics that require understanding whether variables are cointegrated, whether the analysis is univariate or multivariate, and whether structural relationships should be imposed.

Tables A.1 and A.2 in Appendix A provide detailed breakdowns of model performance and per-category accuracy. Appendix C reports API response latencies, though these figures reflect cloud-hosted inference and do not directly apply to local deployment scenarios.

Extended robustness evaluation

The single-turn evaluation on curated prompts establishes baseline tool selection accuracy but leaves several questions unanswered: How do models perform across multi-turn conversations? Do they handle naturalistic language with typos and informal phrasing? Can they extract parameters correctly and interpret results accurately? To address these gaps, we conducted an extended evaluation across five dimensions using the same 10 models.

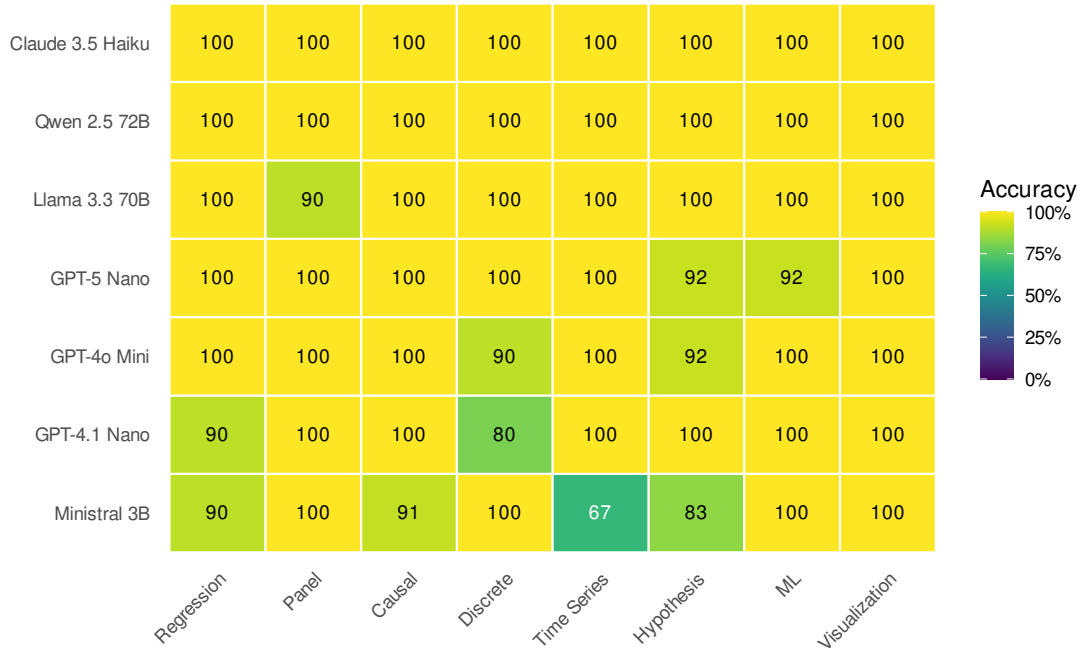

Figure 9: Accuracy breakdown by test category. Darker cells indicate lower accuracy. Most models achieve 100% on standard categories; errors concentrate in edge cases involving ambiguous method selection (e.g., logit with fixed effects vs. FEGLM).

Multi-turn conversations. We constructed 20 multi-turn conversations totaling 72 turns, where each turn builds on previous context (e.g., "Now use robust standard errors instead"). Accuracy drops substantially compared to single-turn: the best-performing model (Llama 3.3 70B) achieves 59.7% turn accuracy and 35% conversation completion (all turns correct). Most models cluster between 47–58% turn accuracy. This decline reflects the challenge of maintaining context—models frequently lose track of prior specifications or misinterpret references to earlier results.

Naturalistic prompts. We tested 99 prompts featuring realistic variations: typos ("regresion"), informal language ("run a quick OLS"), domain jargon ("estimate the ATT"), verbose descriptions, and ambiguous phrasing. Accuracy ranges from 58–70%, with Claude 3.5 Haiku achieving the highest robustness (69.7%). Models handle typos well (65–69% accuracy) but struggle more with ambiguous specifications (42–58%).

Parameter extraction. Beyond tool selection, LLMs must extract variable names, options, and modifiers. We evaluated 49 parameter extraction tasks, measuring precision, recall, and F1 score. Average F1 ranges from 41–59% across models, with Qwen 2.5 72B achieving the highest score (59.1%). Common errors include omitting optional parameters, misidentifying column names from context, and confusing similar parameter types.

Interpretation accuracy. We evaluated whether models correctly interpret statistical output across 26 test cases covering coefficient interpretation, significance testing, and diagnostic results. Models achieve 68–87% accuracy, with GPT-4.1 Nano performing best (87.2%). Errors typically involve misinterpreting log-transformed coefficients or overlooking important

caveats about statistical significance.

Out-of-scope detection. When users request methods outside the implemented set (e.g., Bayesian inference, GARCH models), the system should either decline gracefully or suggest alternatives. Detection rates range from 40–60%, with most undetected cases resulting in selection of a superficially similar but incorrect method.

Table B.1 in Appendix B provides complete results. Figure B.1 visualizes the multi-dimensional performance profile for each model.

Implications. The extended evaluation reveals that single-turn accuracy substantially overstates real-world capability. Multi-turn accuracy of 50–60% means users should expect to correct or clarify requests frequently. Parameter extraction errors occur in roughly half of interactions. These findings reinforce the importance of the conversation transcript and result verification discussed below. For production deployment, larger models (70B+ parameters) provide meaningful improvements in robustness, though even the best models require user oversight for complex analytical workflows.

7.4. Failure modes and error handling

Understanding system behavior under failure conditions is essential for informed use. Three categories of failure merit discussion.

Method misselection

When the LLM selects an inappropriate statistical method, the system proceeds with the selected method and returns results. Users must recognize the mismatch between request and execution. Several patterns characterize such errors:

- *Ambiguous specifications*: A request for "regression with firm effects" could invoke OLS with dummy variables, within-transformation fixed effects, or random effects. The LLM chooses based on training patterns, which may not match user intent.
- *Similar method confusion*: Requests involving VAR/VECM/VARMA, logit/probit/FEGLM, or IPW/AIPW may trigger closely related but distinct methods.
- *Capability boundaries*: Requests for methods outside the implemented set may map to superficially similar available methods rather than returning "not supported."

Users can mitigate misselection by specifying methods explicitly ("use a within-transformation fixed effects model") rather than describing goals abstractly. The system's tool call transparency allows inspection of which method was actually invoked before results are generated.

Estimation failures

When statistical estimation fails—due to singular matrices, non-convergence, or invalid specifications—the `EconError` type returns structured error information that the LLM can interpret and explain. Common cases include:

- *Perfect multicollinearity*: The system detects rank-deficient design matrices and reports which columns are linearly dependent.

- *Convergence failures*: Iterative methods (logit, probit, ARIMA) report iteration count, last parameter change, and suggestions for alternative specifications.
- *Insufficient variation*: Panel estimators detect cases with too few entities or time periods for the requested specification.
- *Small cluster warnings*: Clustered standard errors with fewer than 10 clusters trigger warnings about finite-sample bias.

These errors enable the LLM to provide actionable guidance ("try removing the collinear variable" or "consider robust standard errors instead of clustering") rather than generic failure messages.

Parameter extraction errors

Even when the correct method is selected, the LLM may extract incorrect parameters from the user's request. Common patterns include:

- *Variable name mismatches*: The LLM may infer variable names that do not exist in the dataset, triggering column-not-found errors.
- *Specification omissions*: Options like robust standard errors or clustering variables may not be extracted from ambiguous phrasing.
- *Incorrect defaults*: The LLM may apply default specifications (e.g., including an intercept) when the user intended otherwise.

The dataset schema—column names and types—is provided to the LLM as context, which substantially reduces variable name errors. For specification options, explicit phrasing ("with HC1 robust standard errors") yields more reliable extraction than implicit expectations.

Categorical variable handling

Categorical (string) variables are automatically converted to dummy variables using treatment coding (one level omitted as reference). The system:

- Detects string columns and creates indicator variables for each unique level, dropping the first alphabetically as reference.
- Reports the reference category in output tables.
- Warns when categorical variables have many levels (> 50), as this inflates the design matrix and may cause memory issues.
- Handles missing categories (levels in test data not seen in training) by raising explicit errors rather than producing undefined behavior.

Users can specify a different reference category via the `ref_level` parameter. For ordered categorical variables (ordinal data), the system treats them as unordered by default; polynomial contrasts require explicit specification.

Section 4.5 demonstrates error handling behavior through worked examples showing multicollinearity detection, convergence failures, small-cluster warnings, and out-of-scope request handling.

7.5. Deployment recommendations

The evaluation results inform practical recommendations for local deployment, summarized in Table 6. The most striking finding is that open-source models can match proprietary alternatives: Qwen 2.5 72B achieves 100% accuracy on our benchmark, identical to Claude 3.5 Haiku. This demonstrates that users requiring complete data isolation need not sacrifice tool selection quality—fully local deployments can perform equivalently to cloud-hosted solutions.

Table 6: Recommended models for local deployment. Accuracy figures are from our 87-test benchmark. Storage refers to disk space for the quantized model file using Q4_K_M quantization (4-bit with medium dequantization quality), which provides a good balance between size and accuracy. RAM is memory required during inference, typically 1.5–2× storage due to KV cache and computational buffers. Higher quantization levels (Q5, Q6, Q8) improve accuracy slightly but require proportionally more storage and RAM.

Model	Params	Storage	RAM	Acc. (%)	Notes
High accuracy, workstation hardware					
Qwen 2.5 72B	72B	42 GB	48 GB	100.0	Matches proprietary models
Llama 3.3 70B	70B	40 GB	48 GB	98.8	Strong open-source option
Balanced accuracy, consumer hardware					
Llama 3.1 8B	8B	4.7 GB	8 GB	—	BFCL benchmark leader
Qwen 2.5 7B	7B	4.4 GB	8 GB	—	Strong tool-calling accuracy
Viable accuracy, minimal hardware					
Ministral 3B	3B	2.0 GB	6 GB	90.8	Smallest viable model

At the other end of the spectrum, Ministral 3B demonstrates that even very small models can achieve viable accuracy for tool selection. With only 3 billion parameters, this model runs on virtually any modern laptop—approximately 6 GB VRAM for GPU inference, or CPU-only with 8 GB RAM. The 9.2% error rate concentrates in genuinely ambiguous cases such as distinguishing between VAR and VECM specifications or selecting among discrete choice models with different fixed effects structures. For routine analytical tasks, this accuracy level may be entirely acceptable, particularly given the substantial reduction in hardware requirements.

Between these extremes, the 7–8B parameter class offers a practical middle ground. Models in this range fit comfortably on consumer GPUs with 8+ GB VRAM (e.g., NVIDIA RTX 3070/4070) while providing strong tool-calling performance on established benchmarks. Apple M-series devices with unified memory can also run these models, though memory is shared between CPU, GPU, and system functions; a device with 16 GB unified memory provides roughly equivalent capacity to a dedicated 8 GB GPU for model inference. The 70B+ models, by contrast, require either professional GPUs with 24+ GB VRAM, multi-GPU configurations, or partial CPU offloading that significantly increases inference latency. Users with limited RAM can configure Ollama to offload model layers to disk, trading response time for

reduced memory consumption.

7.6. Privacy guarantees

The fully local configuration provides complete data isolation. No network requests are made to external services—all model inference occurs on the user's hardware, dataset contents never leave the machine, and query history remains in local storage. This architecture is suitable for analysis of sensitive data where regulatory requirements (such as GDPR, HIPAA, or institutional review board protocols) or organizational policy prohibit cloud processing. Users deploying this configuration should verify that their Ollama installation does not enable telemetry or automatic model updates, which could initiate network connections during analysis sessions.

8. Conclusion

This paper has introduced chat-first data analytics as an approach to statistical computation that combines natural language interfaces with pre-implemented, verified statistical methods. The key insight is that language models excel at interpreting user intent and selecting appropriate methods, but should not be trusted with numerical computation. By separating interpretation from execution, chat-first systems can provide the accessibility of conversational interfaces while maintaining the reliability that scientific work demands.

prompt2analytics demonstrates this approach through a foundational econometrics library implemented in pure Rust, exposed through the Model Context Protocol for integration with language models. The software makes several contributions:

1. **A foundational econometrics library for Rust.** The **p2a-core** library provides core methods across regression analysis, panel data econometrics, instrumental variables, causal inference, discrete choice, and time series analysis—filling a gap in the Rust ecosystem for applied econometrics.

2. **Validated implementations.** Numerical accuracy is verified against established R packages across thousands of test cases, with tolerances calibrated to sample size. This addresses concerns about numerical accuracy in statistical software raised by McCullough (1997).

3. **Performance advantages.** For computationally intensive methods (panel fixed effects, robust standard errors, clustering), Rust implementations provide 3–6× speedups over equivalent R code.

4. **Local LLM viability.** Our evaluation demonstrates that even small models (3B parameters) can achieve over 90% accuracy on tool selection, enabling fully local deployment for privacy-sensitive applications.

5. **MCP as an integration standard.** The software demonstrates that MCP provides a practical foundation for LLM-assisted scientific computing, separating concerns between language understanding and numerical computation.

8.1. Limitations

LLM interpretation failures. The 9.2% error rate with the smallest model concentrates in ambiguous cases; safeguards (result inspection, reformulation, clarifying questions) require statistical knowledge to be effective.

Evaluation scope. Single-turn accuracy substantially overstates practical capability. Extended evaluation shows multi-turn accuracy of 50–60%, parameter extraction F1 of 41–59%, and out-of-scope detection rates of 40–60% (Appendix B). User study validation remains unaddressed.

Method coverage. Unimplemented methods include quantile regression, Heckman selection, spatial econometrics, GMM, dynamic panel models, matching estimators, and Bayesian methods. See Section 2 for the complete list.

Scalability. Data are loaded entirely into memory; datasets exceeding available RAM require external preprocessing.

Architecture. Uses embedded SurrealDB for persistence (zero-configuration but not suitable for external database integration), 64-bit precision throughout, and semantic versioning for the MCP schema (currently 0.x, API may evolve).

8.2. Future directions

Several directions warrant further development.

User studies. Controlled experiments measuring task completion rates, time-to-insight, and error detection rates would provide stronger evidence of the chat-first paradigm's practical benefits. Such studies should compare chat-first interfaces against traditional code-based workflows across users with varying levels of statistical expertise.

Enhanced error detection. The system could proactively identify potential issues before estimation: detecting perfect multicollinearity, warning about small cluster counts, suggesting appropriate standard error corrections based on data structure. Current implementations provide errors after failures rather than preventive guidance.

Formal verification. Property-based testing and formal verification techniques could strengthen confidence in implementation correctness beyond comparison to reference implementations. QuickCheck-style testing for statistical properties (e.g., residuals orthogonal to regressors) would complement numerical accuracy validation.

Method expansion. High-priority additions include Bayesian inference (providing posterior distributions rather than point estimates), quantile regression, sample selection corrections, and spatial econometric methods. The modular architecture facilitates these extensions.

Workflow integration. Integration with R and Python via foreign function interfaces would enable users to call Rust implementations from familiar environments, combining the performance benefits of compiled code with existing analytical workflows.

8.3. Broader implications

Chat-first data analytics represents one instance of a broader pattern: using language models as interpreters between human intent and specialized software. This pattern applies beyond econometrics to any domain where users express goals in natural language and systems execute well-defined operations.

The success of this approach depends on a clear division of labor. Language models interpret and explain; specialized software computes. Attempts to have language models perform both roles—generating code that computes statistical quantities—conflate interpretation with execution and sacrifice the reliability that scientific work requires.

The viability of local LLM deployment demonstrated in Section 7 has implications beyond data privacy. If small models can reliably select among pre-implemented methods, the computational requirements for LLM-assisted analysis become modest enough for widespread deployment. This democratizes access to conversational interfaces without requiring cloud connectivity or powerful hardware.

Finally, the design of **prompt2analytics** suggests a template for scientific software in the LLM era: implement algorithms in performant, memory-safe languages; expose them through structured protocols; and let language models handle the translation between human intent and computational execution. This separation of concerns preserves the benefits of both paradigms while mitigating their respective weaknesses.

Acknowledgments

The authors thank the reviewers for constructive feedback that improved the paper.

Appendix

Contents

A. LLM evaluation details

This appendix provides detailed results from the tool selection evaluation described in Section 7. Table A.1 presents accuracy, cost, and failure rates across all evaluated models. Table A.2 breaks down accuracy by test category.

Table A.1: LLM tool selection accuracy and cost comparison. Models marked with † had partial evaluations due to API rate limits or transient server errors during the evaluation period; accuracy figures for these models should be interpreted with caution given the smaller sample sizes. Cost estimates are based on approximately 11,000 input tokens (tool schema) plus 75 tokens (prompt/response) per request at January 2026 API pricing; these figures are subject to rapid change as providers adjust pricing. Local deployment has zero marginal API cost.

Model	Size	Provider	N	Exact	Fail	Acc. (%)	Cost ($)
Claude 3.5 Haiku	—	Anthropic	87	87	0	100.0	2.65
Qwen 2.5 72B	72B	OpenRouter	87	87	0	100.0	2.63
Nemotron 9B†	9B	OpenRouter	43	43	0	100.0	0.22
Llama 3.3 70B	70B	OpenRouter	87	86	0	98.8	1.17
DeepSeek†	—	OpenRouter	65	64	1	98.5	1.11
GPT-4o Mini	—	OpenAI	87	85	1	97.7	1.47
GPT-5 Nano	—	OpenAI	87	85	1	97.7	4.91
GPT-4.1 Nano	—	OpenAI	87	84	2	96.5	0.98
Ministral 3B	3B	OpenRouter	87	79	2	90.8	0.39
Qwen3 8B†	8B	OpenRouter	26	24	2	88.5	0.13

B. Extended robustness evaluation

This appendix provides detailed results from the extended evaluation described in Section 7. The evaluation assesses model performance across five dimensions beyond single-turn tool se-

Table A.2: Accuracy by test category (%). Each category contains 8–13 test cases covering basic to advanced use cases. Bold row shows overall accuracy across all 87 test cases.

Category	Haiku	Qwen	Llama	4o-mini	5-nano	4.1-nano	Mini-3B
Regression	100.0	100.0	100.0	100.0	100.0	90.0	90.0
Panel Data	100.0	100.0	90.0	100.0	100.0	100.0	100.0
Causal Inference	100.0	100.0	100.0	100.0	100.0	100.0	90.9
Discrete Choice	100.0	100.0	100.0	90.0	100.0	80.0	100.0
Time Series	100.0	100.0	100.0	100.0	100.0	100.0	66.7
Hypothesis Testing	100.0	100.0	100.0	91.7	91.7	100.0	83.3
Machine Learning	100.0	100.0	100.0	100.0	91.7	100.0	100.0
Visualization	100.0	100.0	100.0	100.0	100.0	100.0	100.0
Overall	100.0	100.0	98.9	97.7	97.7	96.6	90.8

lection: multi-turn conversations, naturalistic prompts, parameter extraction, interpretation accuracy, and out-of-scope detection.

B.1. Evaluation methodology

Multi-turn conversations. Twenty conversations with 3–4 turns each (72 total turns) simulate realistic analytical workflows where subsequent requests build on prior context. Each turn is scored independently; a conversation is "complete" only if all turns are correct.

Naturalistic prompts. Ninety-nine prompts across eight categories include realistic variations: typos ("regresion with robust se"), informal language ("just run a quick ols"), domain jargon ("estimate the ATT using DID"), verbose descriptions, and ambiguous specifications. Prompts are classified by variation type to identify specific robustness gaps.

Parameter extraction. Forty-nine test cases evaluate whether models correctly extract variable names, options (robust SE type, clustering variables), and modifiers from natural language. We compute precision (correct extractions / total extractions), recall (correct extractions / expected parameters), and F1 score.

Interpretation accuracy. Twenty-six statistical outputs spanning coefficient interpretation, hypothesis test results, and diagnostic statistics are presented to each model with interpretation prompts. Responses are scored for coverage of expected elements and absence of common errors (e.g., confusing correlation with causation, misinterpreting log coefficients).

Out-of-scope detection. Twenty prompts request methods outside the **prompt2analytics** scope (Bayesian inference, GARCH, quantile regression, spatial econometrics, etc.). Correct responses either decline the request or suggest the closest available alternative; incorrect responses select an inappropriate tool.

B.2. Results

Table B.1 summarizes performance across all dimensions. The multi-turn and parameter extraction results reveal substantially lower accuracy than single-turn tool selection, highlighting the gap between controlled evaluation and realistic usage patterns.

Figure B.1 visualizes performance profiles, revealing that no single model dominates across all dimensions. Claude 3.5 Haiku shows strong single-turn and naturalistic performance but

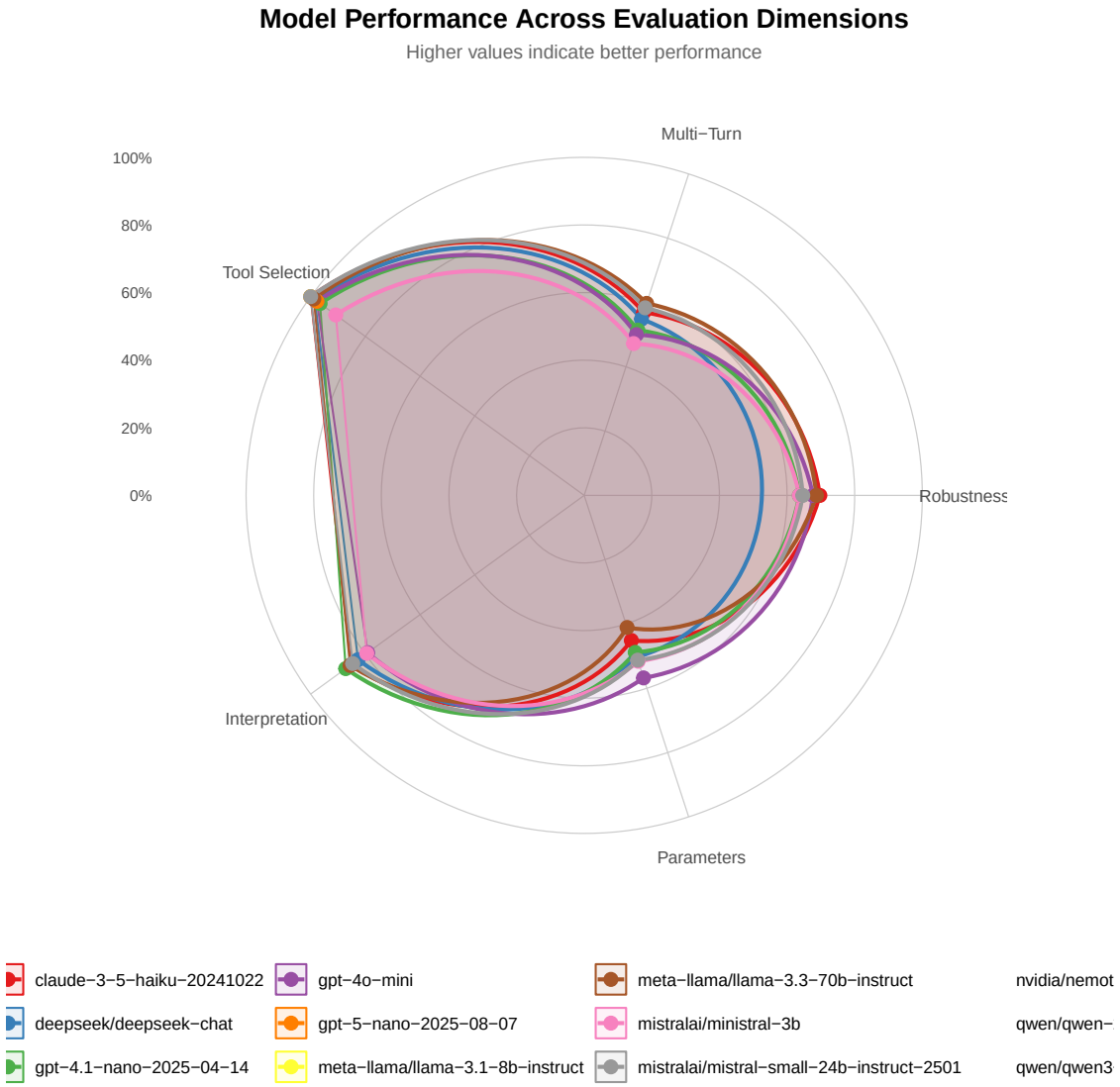

Figure B.1: Multi-dimensional performance profile by model. Each axis represents a robustness dimension normalized to 0–100%. Models cluster in the 50–70% range for most dimensions, with interpretation accuracy showing the strongest performance and out-of-scope detection the weakest.

Table B.1: Extended evaluation results across robustness dimensions. Single-Turn shows baseline tool selection accuracy (87 tests). Multi-Turn shows turn-level accuracy (72 turns); Conv. Compl. shows percentage of conversations with all turns correct (20 conversations). Naturalistic shows accuracy on informal prompts (99 tests). Param F1 shows parameter extraction F1 score (49 tests). Interpret. shows interpretation accuracy (26 tests). OOS Det. shows out-of-scope detection rate (20 tests). Dashes indicate incomplete evaluation data due to API errors.

Model	Single-Turn	Multi-Turn	Conv. Compl.	Natural-istic	Param F1	Inter-pret.	OOS Det.
Claude 3.5 Haiku	100.0	56.9	25.0	69.7	45.1	85.3	45.0
GPT-4o-mini	97.7	50.0	10.0	67.7	56.8	75.6	55.0
GPT-4.1 Nano	96.6	51.4	20.0	63.6	48.8	87.2	50.0
Llama 3.3 70B	98.9	59.7	35.0	68.7	41.1	85.6	55.0
Qwen 2.5 72B	100.0	56.9	15.0	68.7	59.1	82.7	—
DeepSeek Chat	98.5	54.8	0.0	—	50.3	82.7	—
Mistral Small 24B	100.0	58.3	30.0	64.6	51.2	84.6	55.0
Ministral 3B	90.8	47.2	20.0	63.6	51.6	79.5	40.0
Qwen3 8B	96.2	55.6	25.0	57.6	43.7	67.8	55.0
Nemotron Nano 9B	97.7	48.6	15.0	58.6	51.9	80.3	60.0

moderate multi-turn accuracy. Llama 3.3 70B achieves the best multi-turn coherence. Qwen 2.5 72B leads in parameter extraction. These trade-offs suggest that model selection should consider the expected usage pattern.

Figure B.2 directly compares controlled vs. realistic evaluation conditions. The 30–40 percentage point drop from single-turn to multi-turn accuracy is consistent across models, suggesting a fundamental challenge in maintaining conversational context rather than a model-specific limitation.

B.3. Failure analysis

Common failure patterns across dimensions include:

- *Context drift*: In multi-turn conversations, models frequently "forget" prior specifications, reverting to defaults or misinterpreting pronoun references ("use the same variables" interpreted incorrectly).
- *Partial parameter extraction*: Models often extract the primary parameters (dependent/independent variables) but omit secondary options (robust SE type, clustering specification, interaction terms).
- *Overconfident tool selection*: When facing out-of-scope requests, models rarely decline; instead, they select superficially similar tools (e.g., OLS for quantile regression, VAR for Bayesian VAR).
- *Interpretation shortcuts*: Models sometimes provide accurate but incomplete interpretations, omitting important caveats about statistical vs. practical significance or the limitations of specific tests.

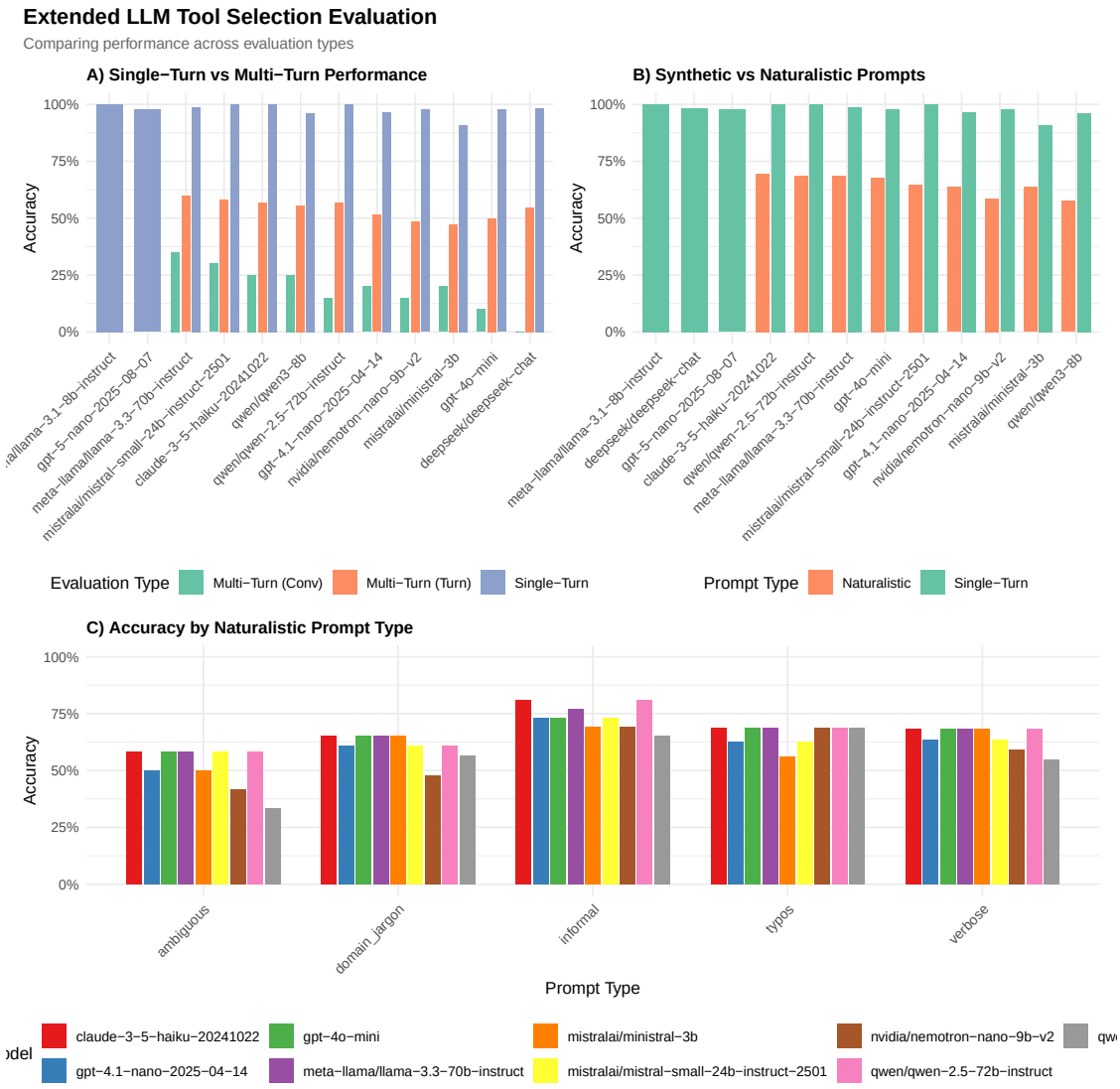

Figure B.2: Comparison of single-turn vs. multi-turn accuracy (left) and synthetic vs. naturalistic prompt robustness (right). The consistent gap between controlled and realistic conditions indicates that baseline evaluations overstate practical capability.

These patterns inform the deployment recommendations in Section 7: explicit specifications reduce context drift, parameter verification catches extraction errors, and result review identifies interpretation gaps.

C. API response latency

Figure C.1 presents response latency distributions for the models evaluated via their respective cloud APIs. These measurements include network round-trip time and reflect API-hosted inference, not local deployment performance.

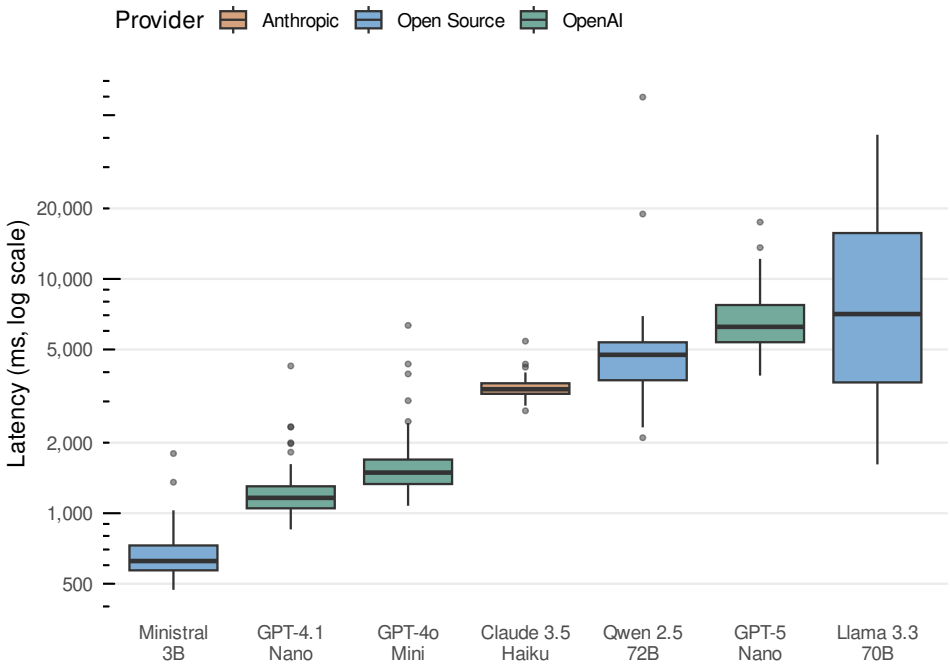

Figure C.1: API response latency distribution by model (log scale). Latencies reflect cloud-hosted inference via OpenAI, Anthropic, and OpenRouter APIs, including network round-trip time. Local deployment latency depends on hardware configuration and would differ substantially from these figures.

For cloud-hosted inference, GPT-4.1 Nano shows the fastest median response (1,245ms) among high-accuracy models, while Ministral 3B achieves 650ms median latency via OpenRouter. These figures are useful for comparing relative API performance but do not predict local inference speed.

Local deployment latency depends on several factors: model size and quantization level, available RAM and GPU VRAM, CPU or GPU compute capability, and context length (which affects KV cache size). As a rough guide, GPU-accelerated inference on consumer hardware (RTX 3070/4070 with 8 GB VRAM) typically achieves 1–3 seconds for 7–8B parameter models. CPU-only inference on modern laptops ranges from 5–15 seconds depending on model size and system specifications. Systematic benchmarks of local deployment latency across hardware configurations represent an important direction for future work; the cloud

API measurements here serve primarily to compare relative performance across models rather than to predict local inference speed.

D. Deployment guide

This appendix provides deployment instructions for various platforms.

D.1. Linux workstation

1. Install Rust (1.85+): `curl -proto '=https' -tlsv1.2 -sSf https://sh.rustup.rs | sh`
2. Clone repository: `git clone https://github.com/prompt2analytics/prompt2analytics`
3. Build MCP server: `cargo build -release -p p2a-mcp`
4. Install Ollama: `curl -fsSL https://ollama.ai/install.sh | sh`
5. Pull a model: `ollama pull llama3.1:8b`
6. Run MCP server: `./target/release/p2a-mcp -transport stdio`
7. Configure Claude Desktop or other MCP client to use the server

D.2. macOS (Apple Silicon)

Apple M-series chips provide excellent performance for local LLM inference due to unified memory architecture. Recommended configuration:

- M1/M2: 7–8B models with Q4_K_M quantization
- M2 Pro/Max: 13B models or 8B at Q8 quantization
- M3 Max/Ultra: Up to 70B models with sufficient unified memory

Build commands are identical to Linux. Homebrew can simplify dependency management: `brew install rust ollama`.

D.3. Windows (WSL2)

Native Windows builds are supported but WSL2 provides a more consistent experience:

1. Install WSL2: `wsl -install -d Ubuntu`
2. Follow Linux instructions within WSL2
3. For GPU acceleration, install NVIDIA CUDA drivers on Windows (WSL2 will use them automatically)

D.4. MCP client configuration

For Claude Desktop, add to `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "prompt2analytics": {
      "command": "/path/to/p2a-mcp",
      "args": ["--transport", "stdio"]
    }
  }
}
```

D.5. Model selection guidance

Use Case	Recommended Model	Notes
Maximum privacy	Llama 3.1 8B (local)	No data leaves machine
Best accuracy	Claude 3.5 Sonnet	Requires API key
Cost-sensitive	GPT-4.1 Nano	Cheapest high-accuracy option
Resource-constrained	Ministral 3B	Runs on 8GB RAM

E. Privacy and security

This appendix documents data transmission behavior and security considerations.

E.1. Data transmission summary

Table E.1 lists operations that may transmit data beyond the local machine when using cloud LLM providers.

Table E.1: Operations that may transmit data to LLM providers

Operation	Data Transmitted	Risk Level
Dataset load	Column names, types, row count, null percentages	Low
Data preview (head/tail/sample)	Actual row values	High
Data profiling	Frequent values, min/max, examples	Medium
Statistical estimation	Coefficients, SEs, test statistics only	Low
Outlier detection	Specific observation values	Medium
Error messages	May include problematic values	Low

E.2. Audit logging

To enable audit logging of all MCP tool calls:

```
p2a-mcp --transport stdio --log-level debug --log-file audit.log
```

The log records: timestamp, tool name, parameters (excluding bulk data), and result summary. Review logs to verify no unexpected data transmission.

E.3. Network verification

To verify local-only operation with Ollama:

1. Start Ollama: `ollama serve`
2. Block outbound connections: `sudo iptables -A OUTPUT -j DROP`
3. Run analysis—should complete successfully
4. Restore network: `sudo iptables -D OUTPUT -j DROP`

E.4. Security best practices

- Avoid loading untrusted data files (potential injection payloads)
- Review tool calls when processing sensitive data
- Use local LLM deployment for high-security environments
- Keep software updated; security patches are released as needed
- The MCP server has no filesystem access beyond loaded datasets

F. Reproducibility

This appendix provides detailed instructions for reproducing the results in this paper.

F.1. Version information

Results in this paper were generated with the following software versions:

- Rust: 1.85.0 (stable)
- R: 4.5.0 with OpenBLAS
- Key R packages: **plm** 2.6-4, **lfe** 3.1-0, **sandwich** 3.1-1, **fixest** 0.12.1, **forecast** 8.23.0
- Operating system: Ubuntu 24.04 LTS (Linux 6.8)

F.2. Reproducing benchmarks

Performance benchmarks can be reproduced using the following protocol:

1. Install R 4.5+ with required packages: **plm**, **lfe**, **sandwich**, **forecast**, **microbenchmark**
2. Build **prompt2analytics** in release mode: `cargo build -release -p p2a-cli`
3. Execute benchmarks: `make -C paper/code benchmarks`

Both languages use seed = 42 for random number generation. The benchmark protocol includes 10 warmup iterations before 100 measured iterations for fast methods (50 iterations for moderate, 20 for slow). Detailed instructions are provided in `paper/code/README_REPRODUCIBILITY.md`.

F.3. Containerized reproduction

A `Dockerfile` in the repository root provides a containerized environment with all dependencies pinned to the versions above:

```
docker build -t p2a-paper .
docker run -v $(pwd)/paper:/output p2a-paper make all
```

The container includes both R and Rust toolchains with identical configurations to those used for this paper.

G. Benchmark methodology

This appendix provides complete details on the benchmark methodology used in Sections 5 and 6.

G.1. Data generating processes

Benchmark datasets are generated synthetically using identical data-generating processes in both languages with fixed random seeds (seed = 42) for exact reproducibility.

Regression benchmarks. Data follow $y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i$ with true parameters $\beta_0 = 1.0$, $\beta_1 = 2.0$, $\beta_2 = -1.5$, and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ where $\sigma = 1.0$. Covariates are drawn as $x_{1i} \sim \mathcal{N}(0, 1)$ and $x_{2i} \sim \mathcal{N}(0, 1)$ with correlation $\rho = 0.3$.

Panel data benchmarks. Entity effects $\alpha_i \sim \mathcal{N}(0, 0.5^2)$ are added, with 100 entities observed over 10 periods.

Discrete choice benchmarks. The latent index $y_i^* = \beta' x_i + \varepsilon_i$ is transformed via the logit or probit link.

Time series benchmarks. Data follow ARIMA$(1, 1, 1)$ with $\phi = 0.7$, $\theta = 0.3$, and innovation variance $\sigma_\eta^2 = 1.0$.

Complete DGP specifications are provided in `paper/code/README_REPRODUCIBILITY.md`.

G.2. Benchmark execution protocol

Each benchmark configuration executes multiple iterations after a warmup phase:

- 100 iterations for fast methods (OLS, robust SEs, PCA)

- 50 iterations for moderate methods (panel FE, clustering)
- 20 iterations for slow methods (ARIMA, MSTL decomposition)

R benchmarks use the **bench** package, which handles garbage collection timing. Rust benchmarks use Criterion with automatic warmup and outlier detection. We report full timing distributions (minimum, quartiles, maximum, mean, standard deviation) rather than point estimates alone.

G.3. Hardware and software environment

All benchmarks run on identical hardware: an Intel Core i7-1260P processor with 64GB RAM running Linux, with R 4.5.2 using OpenBLAS and Rust 1.92.0 compiled in release mode with link-time optimization.

H. Related work: Statistical interfaces

This appendix provides extended discussion of related work on statistical computing interfaces.

H.1. Formula notation and domain-specific languages

The most influential interface innovation in statistical computing was the Wilkinson-Rogers formula notation (Wilkinson and Rogers 1973), which introduced the symbolic representation of linear models that remains central to R, Stata, and other systems (e.g., `y ~ x1 + x2`). Chambers and Hastie (1992) extended this notation in S and later R, establishing design principles for statistical software that prioritize expressiveness and composability (Chambers 1998, 2008). The formula interface represents a domain-specific language (DSL) for model specification—a pattern that Stata adopts with its command syntax and that appears in specialized systems like Stan's probabilistic programming language (Carpenter, Gelman, Hoffman, Lee, Goodrich, Betancourt, Brubaker, Guo, Li, and Riddell 2017).

prompt2analytics takes a fundamentally different approach: rather than inventing new notation, it accepts natural language descriptions of analytical intent. This trades the precision of formal DSLs for accessibility, relying on LLMs to bridge the gap between informal specification and structured execution.

H.2. Natural language interfaces to data systems

Natural language interfaces (NLIs) for data systems have a long history. LUNAR (Woods 1973) demonstrated feasibility for database querying; subsequent decades produced various NLI systems with limited adoption (Androutsopoulos *et al.* 1995). Commercial systems attempted similar approaches: SPSS's syntax assistance, SAS Enterprise Guide's point-and-click interface with natural language elements, and JMP's scripting language with English-like constructs.

The text-to-SQL paradigm, revitalized by deep learning, represents recent progress on natural language data interfaces. Benchmarks like WikiSQL (Zhong, Xiong, and Socher 2017) and Spider (Yu *et al.* 2018) have driven substantial improvements in translating natural language

to structured queries (Li *et al.* 2024). NL4DV (Narechania *et al.* 2021) enables natural language specification of visualizations.

H.3. Automated machine learning

AutoML systems automate model selection through algorithmic search. Auto-WEKA (Thornton, Hutter, Hoos, and Leyton-Brown 2013), Auto-sklearn (Feurer *et al.* 2015), and H2O AutoML (LeDell and Poirier 2020) optimize predictive performance over predefined pipeline spaces. Recent work integrates LLMs into AutoML: Hollmann, Müller, and Hutter (2024) use language models to guide feature engineering and hyperparameter search.

AutoML differs from chat-first analytics in objective and scope. AutoML optimizes for prediction; chat-first analytics interprets user intent across prediction, causal inference, hypothesis testing, and description.

H.4. Computational econometrics

The computational econometrics literature has established standards for numerical accuracy and implementation quality. McCullough (1997) and McCullough and Vinod (1999) documented numerical inaccuracies across commercial and open-source statistical packages. Textbooks by Davidson and MacKinnon (2004) and Cameron and Trivedi (2005) provide authoritative treatments of econometric methods and their implementation.

I. Implementation notes

This appendix documents implementation caveats and limitations for specific methods.

I.1. Instrumental variables

The 2SLS implementation reports first-stage F-statistics for instrument relevance. The traditional rule-of-thumb ($F > 10$) from Staiger and Stock (1997) has been superseded by more nuanced guidance. Stock and Yogo (2005) provide critical values depending on the number of instruments and acceptable bias levels; Lee, McCrary, Moreira, and Porter (2022) offer further refinements for inference under weak identification. The current implementation does not provide Stock-Yogo critical values or weak-identification-robust inference methods (Anderson-Rubin, conditional likelihood ratio). Users with potentially weak instruments should verify results using R's **ivmodel** or **AER** packages.

I.2. Difference-in-differences

The DiD implementation assumes the canonical two-group, two-period design. Recent econometric literature has documented severe biases when this estimator is applied to staggered treatment timing. Goodman-Bacon (2021) demonstrates that two-way fixed effects DiD produces a weighted average of all possible 2×2 comparisons, including problematic "forbidden" comparisons using already-treated units as controls. de Chaisemartin and D'Haultfoeuille (2020) and Sun and Abraham (2021) propose alternative estimators robust to treatment effect heterogeneity.

prompt2analytics does not implement staggered DiD estimators. Users with staggered treatment adoption should use **fixest**'s `sunab()` function, the **did** package, or **did2s** in R.

I.3. Random effects

The random effects estimator uses the Swamy-Arora variance component estimator, which can produce negative variance estimates for σ_α^2 (the between-entity variance) in certain data configurations. When this occurs, the implementation truncates the estimate to zero and proceeds with pooled OLS, issuing a warning.

I.4. Discrete choice models

Logit and probit models report coefficients on the latent index scale. For interpretation, marginal effects are computed as *Average Marginal Effects* (AME): the partial derivative $\partial \Pr(y = 1)/\partial x_j$ is evaluated at each observation's covariate values, then averaged across the sample. This differs from *Marginal Effects at Means* (MEM), which evaluates derivatives at the sample mean of all covariates. AME is generally preferred because it does not depend on potentially unrepresentative "average" covariate values (Cameron and Trivedi 2005).

I.5. Time series models

ARIMA estimation supports non-seasonal models: ARIMA(p, d, q) with automatic or user-specified orders. Seasonal ARIMA (SARIMA) is partially implemented: seasonal differencing and seasonal AR/MA terms can be specified, but automatic seasonal order selection is not available.

VAR and VECM implementations support lag selection via AIC/BIC and Johansen cointegration testing, but do not implement structural VAR identification.

I.6. Methods not implemented

The current implementation focuses on foundational methods. Unimplemented categories include: sample selection (Heckman), quantile regression, spatial econometrics, GMM and structural estimation, dynamic panel models (Arellano-Bond, Arellano-Bover), matching estimators, synthetic control, user-facing bootstrap inference, Bayesian methods, and nonparametric methods.

J. Extended examples

This appendix provides additional worked examples demonstrating **prompt2analytics** capabilities.

J.1. Multi-step workflow example

Real-world analysis typically involves multiple steps beyond single-command estimation. This section demonstrates a realistic workflow including data inspection, handling issues, and specification refinement.

Initial exploration

User: I have employee salary data in salaries.csv. Load it and tell me what variables are available for analysis.

```
**Dataset loaded: salaries**
- Rows: 1,847
- Columns: 8

| Column     | Type    | Non-null | Description                    |
|------------|---------|----------|--------------------------------|
| employee_id| Int64   | 100%     | Unique identifier              |
| salary     | Float64 | 98.2%    | Annual salary (USD)            |
| education  | String  | 99.5%    | Highest degree (HS/BA/MA/PhD)  |
| experience | Float64 | 97.8%    | Years of experience            |
| department | String  | 100%     | Department name                |
| gender     | String  | 100%     | M/F                            |
| performance| Float64 | 94.1%    | Performance rating (1-5)       |
| tenure     | Int64   | 100%     | Years at company               |

I notice some missing values, particularly in performance (5.9% missing)
and experience (2.2% missing). Would you like me to examine the missingness
pattern before proceeding?
```

Addressing data issues

User: Yes, check if the missing values are random or if there's a pattern. Then proceed with complete cases for now.

```
**Missingness Analysis**

Performance ratings are more likely to be missing for:
- Recent hires (tenure < 1 year): 23% missing vs 3% for others
- This suggests MNAR (missing not at random) - new employees may not
  yet have performance reviews.

**Proceeding with complete cases:** 1,692 observations (91.6% retained)
```

Initial specification

User: Regress salary on education, experience, and gender. I want to understand the gender pay gap controlling for human capital.

```
**OLS Regression Results** (HC1 robust standard errors)

| Variable         |     Coef |   Std Err |      t | P>|t| |
|------------------|---------:|----------:|-------:|------:|
| (Intercept)      | 42,156   |   1,247   | 33.81  | 0.000 |
| education_BA     |  8,934   |     892   | 10.02  | 0.000 |
| education_MA     | 15,267   |   1,103   | 13.84  | 0.000 |
| education_PhD    | 22,891   |   1,456   | 15.72  | 0.000 |
| experience       |  1,247   |      89   | 14.01  | 0.000 |
| gender_M         |  4,892   |     743   |  6.58  | 0.000 |

R-squared: 0.412
```

Specification refinement

```
User: Add department fixed effects and performance rating.

**Regression with Department Fixed Effects**

| Variable         |     Coef |   Std Err |      t | P>|t| |
|------------------|---------:|----------:|-------:|------:|
| education_BA     |  6,234   |     812   |  7.68  | 0.000 |
| education_MA     | 11,456   |   1,021   | 11.22  | 0.000 |
| education_PhD    | 17,234   |   1,298   | 13.28  | 0.000 |
| experience       |  1,089   |      82   | 13.28  | 0.000 |
| performance      |  3,456   |     312   | 11.08  | 0.000 |
| gender_M         |  2,134   |     698   |  3.06  | 0.002 |

R-squared (within): 0.523
Department FE: 12 groups

Adding department controls and performance reduces the gender coefficient
from \$4,892 to \$2,134 (a 56\% reduction).
```

J.2. Additional CLI examples

The command-line interface provides programmatic access for scripted workflows:

```
# Session-based workflow
p2a --session analysis.json data load grunfeld.csv --name grunfeld
p2a --session analysis.json data describe grunfeld

# Panel data estimation with clustering
p2a --session analysis.json panel fe grunfeld \
    -y inv -x value capital --entity firm
```

```
# Export session to reproducible script
p2a script export analysis.json -o reproduce.sh
```

J.3. Additional error handling examples

Small cluster warnings

```
User: Cluster the standard errors by department.

**Clustered Standard Errors (by department)**

| Variable    |    Coef | Robust SE | Cluster SE |     t | P>|t| |
|-------------|--------:|----------:|-----------:|------:|------:|
| performance |   0.142 |     0.018 |      0.031 |  4.58 | 0.001 |
| tenure      |   0.023 |     0.004 |      0.007 |  3.29 | 0.008 |
| experience  |   0.008 |     0.003 |      0.004 |  2.00 | 0.071 |

**Warning: Small number of clusters (G = 10)**

Clustered standard errors require $G \to \infty$ asymptotics. With only
10 clusters, standard errors may be biased downward.
```

Out-of-scope requests

```
User: Run a Bayesian regression with informative priors.

**Method not available**

Bayesian inference is not currently implemented. Available alternatives:
- OLS with robust standard errors (implemented)
- Ridge/LASSO regularization for coefficient shrinkage (implemented)
- Bootstrap confidence intervals (implemented)

**For Bayesian analysis**, consider: R (rstanarm, brms), Python (PyMC).
```

References

Androutsopoulos I, Ritchie GD, Thanisch P (1995). "Natural Language Interfaces to Databases – An Introduction." *Natural Language Engineering*, **1**(1), 29–81. doi:10.1017/S135132490000005X.

Anthropic (2024). "Model Context Protocol." Accessed: 2024, URL https://modelcontextprotocol.io/.

Baltagi BH (2013). *Econometric Analysis of Panel Data*. 5th edition. Wiley, Chichester.

Bergé L (2018). "Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package **fixest**." *CREA Discussion Papers*, **13**. URL https://CRAN.R-project.org/package=fixest.

Cameron AC, Trivedi PK (2005). *Microeconometrics: Methods and Applications*. Cambridge University Press, Cambridge. doi:10.1017/CBO9780511811241.

Carpenter B, Gelman A, Hoffman MD, Lee D, Goodrich B, Betancourt M, Brubaker M, Guo J, Li P, Riddell A (2017). "Stan: A Probabilistic Programming Language." *Journal of Statistical Software*, **76**(1), 1–32. doi:10.18637/jss.v076.i01.

Chambers JM (1998). *Programming with Data: A Guide to the S Language*. Springer-Verlag, New York. doi:10.1007/978-1-4684-6310-8.

Chambers JM (2008). *Software for Data Analysis: Programming with R*. Springer-Verlag, New York. doi:10.1007/978-0-387-75936-4.

Chambers JM, Hastie TJ (1992). *Statistical Models in S*. Wadsworth & Brooks/Cole, Pacific Grove, CA.

Chen S, Shen K, Ma X (2025). "Econometrics AI Agent." *arXiv preprint arXiv:2506.00856*. URL https://arxiv.org/abs/2506.00856.

Davidson R, MacKinnon JG (2004). *Econometric Theory and Methods*. Oxford University Press, New York.

de Chaisemartin C, D'Haultfoeuille X (2020). "Two-Way Fixed Effects Estimators with Heterogeneous Treatment Effects." *American Economic Review*, **110**(9), 2964–2996. doi:10.1257/aer.20181169.

Feurer M, Klein A, Eggensperger K, Springenberg JT, Blum M, Hutter F (2015). "Efficient and Robust Automated Machine Learning." In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pp. 2962–2970.

Gaure S (2013). "**lfe**: Linear Group Fixed Effects." *The R Journal*, **5**(2), 104–117. URL https://journal.r-project.org/archive/2013/RJ-2013-031/.

Goodman-Bacon A (2021). "Difference-in-Differences with Variation in Treatment Timing." *Journal of Econometrics*, **225**(2), 254–277. doi:10.1016/j.jeconom.2021.03.014.

Google (2025). "Agent2Agent Protocol (A2A)." Announced April 2025; transferred to Linux Foundation, URL https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/.

Greene WH (2018). *Econometric Analysis*. 8th edition. Pearson, New York.

Hamilton JD (1994). *Time Series Analysis*. Princeton University Press, Princeton, NJ.

Hollmann N, Müller S, Hutter F (2024). "Large Language Models for Automated Machine Learning." In *Proceedings of the 2024 Conference on Neural Information Processing Systems (NeurIPS)*. URL https://arxiv.org/abs/2408.14033.

Hong S, Lin Y, Liu B, Liu B, Wu B, Li D, Chen J, Zhang J, Wang J, Zhang L, Zhang L, Yang M, Zhuge M, Guo T, Zhou T, Tao W, Wang W, Tang X, Lu X, Zheng X, Liang X, Fei Y, Cheng Y, Xu Z, Wu C (2025). "Data Interpreter: An LLM Agent For Data Science." In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19796–19821. URL https://arxiv.org/abs/2402.18679.

IBM Research (2025). "Agent Communication Protocol (ACP)." Launched March 2025; merged with A2A under Linux Foundation, URL https://research.ibm.com/projects/agent-communication-protocol.

Korinek A (2025). "AI Agents for Economic Research." *Journal of Economic Literature*. Forthcoming, URL https://www.aeaweb.org/content/file?id=23290.

LeDell E, Poirier S (2020). "H2O AutoML: Scalable Automatic Machine Learning." In *Proceedings of the 7th ICML Workshop on Automated Machine Learning*. URL https://www.automl.org/wp-content/uploads/2020/07/AutoML_2020_paper_61.pdf.

Lee DS, McCrary J, Moreira MJ, Porter J (2022). "Valid t-ratio Inference for IV." *American Economic Review*, **112**(10), 3260–3290. doi:10.1257/aer.20211063.

Li B, Luo Y, Chai C, Li G, Tang N (2024). "The Dawn of Natural Language to SQL: Are We Fully Ready?" *Proceedings of the VLDB Endowment*, **17**(11), 3318–3331. doi:10.14778/3681954.3682024.

Liu W, Huang X, Zeng X, Hao X, Yu S, Li D, Wang S, Gan W, Liu Z, Yu Y, Wang Z, Wang Y, Ning W, Hou Y, Wang B, Wu C, Wang X, Liu Y, Wang Y, Tang D, Tu D, Shang L, Jiang X, Tang R, Lian D, Liu Q, Chen E (2025). "ToolACE: Winning the Points of LLM Function Calling." In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. URL https://openreview.net/forum?id=8EB8k6DdCU.

Ludwig J, Mullainathan S, Rambachan A (2024). "Large Language Models: An Applied Econometric Framework." *NBER Working Paper*, (33344). doi:10.3386/w33344. URL https://www.nber.org/papers/w33344.

McCullough BD (1997). "A Review of **RATS** v4.2: Benchmarking Numerical Accuracy." *Journal of Applied Econometrics*, **12**(2), 181–190. doi:10.1002/(SICI)1099-1255(199703/04)12:2<181::AID-JAE428>3.0.CO;2-V.

McCullough BD, Vinod HD (1999). "The Numerical Reliability of Econometric Software." *Journal of Economic Literature*, **37**(2), 633–665. doi:10.1257/jel.37.2.633.

Michelaki D, Tsagris M (2025). "An Overview of Economics and Econometrics Related R Packages." *Stats*, **8**(4), 85. doi:10.3390/stats8040085.

Narechania A, Srinivasan A, Stasko J (2021). "NL4DV: A Toolkit for Generating Analytic Specifications for Data Visualization from Natural Language Queries." *IEEE Transactions on Visualization and Computer Graphics*, **27**(2), 369–379. doi:10.1109/TVCG.2020.3030378.

Nejjar A, Zacharias L, Stiehle F, Weber I (2024). "LLMs for Science: Usage for Code Generation and Data Analysis." *Journal of Software: Evolution and Process*, **36**(9), e2723. doi:10.1002/smr.2723.

Ollama Inc (2024). *Ollama: Run Large Language Models Locally*. URL https://ollama.ai.

OWASP Foundation (2025). "OWASP MCP Top 10." Security risks for Model Context Protocol implementations, URL https://owasp.org/www-project-mcp-top-10/.

Patil SG, Zhang T, Wang X, Gonzalez JE (2023). "Gorilla: Large Language Model Connected with Massive APIs." *arXiv preprint arXiv:2305.15334*. URL https://arxiv.org/abs/2305.15334.

Pola-rs Contributors (2024). ***polars**: Blazingly Fast DataFrames in Rust and Python*. Version 0.46, URL https://pola.rs/.

Qin Y, Liang S, Ye Y, Zhu K, Yan L, Lu Y, Lin Y, Cong X, Tang X, Qian B, Zhao S, Hong L, Tian R, Xie R, Zhou J, Gerstein M, Li D, Liu Z, Sun M (2024). "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs." *arXiv preprint arXiv:2307.16789*. URL https://arxiv.org/abs/2307.16789.

Qu C, Dai S, Wei X, Xu J, Cai H, Wang S, Yin D, Wen JR (2025). "Tool Learning with Language Models: A Comprehensive Survey of Methods, Pipelines, and Benchmarks." *Vicinagearth*. doi:10.1007/s44336-025-00024-x.

R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL https://www.R-project.org/.

Rust Foundation (2024). "The Rust Programming Language." URL https://www.rust-lang.org/.

Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." *arXiv preprint arXiv:2302.04761*. URL https://arxiv.org/abs/2302.04761.

Staiger D, Stock JH (1997). "Instrumental Variables Regression with Weak Instruments." *Econometrica*, **65**(3), 557–586. doi:10.2307/2171753.

Stock JH, Yogo M (2005). "Testing for Weak Instruments in Linear IV Regression." In DWK Andrews, JH Stock (eds.), *Identification and Inference for Econometric Models: Essays in Honor of Thomas Rothenberg*, pp. 80–108. Cambridge University Press, Cambridge. doi:10.1017/CBO9780511614491.006.

Sun L, Abraham S (2021). "Estimating Dynamic Treatment Effects in Event Studies with Heterogeneous Treatment Effects." *Journal of Econometrics*, **225**(2), 175–199. doi:10.1016/j.jeconom.2020.09.006.

Sun M, Han R, Jiang B, Qi H, Sun D, Yuan Y, Huang J (2025a). "LAMBDA: A Large Model Based Data Agent." *Journal of the American Statistical Association*. doi:10.1080/01621459.2025.2510000. Online first.

Sun M, Han R, Jiang B, Qi H, Sun D, Yuan Y, Huang J (2025b). "A Survey on Large Language Model-Based Agents for Statistics and Data Science." *The American Statistician*. doi:10.1080/00031305.2025.2561140. Online first.

Thornton C, Hutter F, Hoos HH, Leyton-Brown K (2013). "Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms." In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 847–855. doi:10.1145/2487575.2487629.

Wilkinson GN, Rogers CE (1973). "Symbolic Description of Factorial Models for Analysis of Variance." *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, **22**(3), 392–399. doi:10.2307/2346786.

Woods WA (1973). "Progress in Natural Language Understanding: An Application to Lunar Geology." In *Proceedings of the AFIPS National Computer Conference*, pp. 441–450. doi:10.1145/1499586.1499695.

Wooldridge JM (2010). *Econometric Analysis of Cross Section and Panel Data*. 2nd edition. MIT Press, Cambridge, MA.

Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. URL https://arxiv.org/abs/2210.03629.

Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, Ma J, Li I, Yao Q, Roman S, Zhang Z, Radev D (2018). "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task." In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921. doi:10.18653/v1/D18-1425.

Zhong V, Xiong C, Socher R (2017). "Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning." In *arXiv preprint arXiv:1709.00103*. URL https://arxiv.org/abs/1709.00103.

Affiliation:

First Author
Department of Economics
University of Somewhere
Address Line
City, Country
E-mail: author@example.org
URL: https://example.org/

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd